

EFX₀ Allocations with at Most Three Relevant Goods: Existence and a Verified Polynomial-Time Algorithm

Long version

Evan Lin

University of Illinois Urbana-Champaign

Abstract. An allocation of indivisible goods is EFX₀ if no agent envies another agent’s bundle after removing any single good from it, even a good that the envious agent values at zero. We prove that every instance with nonnegative additive valuations in which each agent positively values at most three goods has a complete EFX₀ allocation, and we give a polynomial-time algorithm, K3ALG, that computes one. K3ALG first lets agents leave with a good they value at least as much as all their other remaining goods together. The remaining agents pick goods in a serial dictatorship that serves agents who have lost a good first; the leftover goods fill free slots, and the surplus goes to a single owner, after at most one rotation of picks along a chain of agents. For natural-number values, the correctness of K3ALG and a polynomial bound on its number of operations in an explicit unit-cost model are machine-checked in Lean 4 on the program itself; a bound on bit operations is written, not formalized. The existence results are machine-checked for values in an abstract ordered additive structure, any linearly ordered cancellative commutative monoid; instantiating them at the nonnegative reals $\mathbb{R}_{\geq 0}$ uses the standard fact that $\mathbb{R}_{\geq 0}$ is such a monoid (core Lean has no real numbers). If every agent values exactly three goods, none more than the other two together, there is a complete EFX₀ allocation in which all but at most one agent receive at most two goods. This long version is meant to be read on its own: it explains the approach informally, traces the algorithm on two examples, and gives every proof of the main theorems in full.

Keywords: Fair division · Indivisible goods · EFX · Serial dictatorship · Formal verification · Lean

1 Introduction

1.1 EFX and EFX₀

We allocate a set M of m indivisible goods to a set N of n agents. Agent i has an *additive valuation*: a value $v_i(g) \geq 0$ for each good g , and $v_i(S) = \sum_{g \in S} v_i(g)$ for every set S of goods. An *allocation* X is a function that

assigns every good to exactly one agent; agent i 's *bundle* X_i is the set of goods assigned to i . So the bundles are pairwise disjoint and cover M : every allocation in this paper is *complete*, and no good may be thrown away.

Envy-freeness, $v_i(X_i) \geq v_i(X_j)$ for all agents i and j , can be impossible with indivisible goods; think of one good and two agents who both value it. *Envy-freeness up to any good* weakens it [4]: agent i may envy the bundle X_j , but the envy must disappear as soon as any single good is removed from X_j . Two versions of this idea are in use. They differ in which goods may be removed:

EFX: $v_i(X_i) \geq v_i(X_j \setminus \{h\})$ for all $i \neq j$ and $h \in X_j$ with $v_i(h) > 0$;

EFX₀: $v_i(X_i) \geq v_i(X_j \setminus \{h\})$ for all $i \neq j$ and $h \in X_j$.

Both conditions are vacuous when X_j is empty. EFX₀ implies EFX, and the two coincide when all values are positive. Whether every instance with additive valuations has a complete EFX allocation is a central open question of fair division [4]. Most known existence results (Section 1.3) are proved in the stronger form EFX₀. The one rule that separates the two notions, and that this paper is about, is the following.

EFX and EFX₀ differ only in which goods may be removed from the other bundle. *Under EFX₀, if another agent's bundle contains a good that agent i values at 0, then i must not envy that bundle at all.*

Example 1 (EFX, but not EFX₀). Two agents share three goods a, b, c . Agent 1 values a at 3, b at 2 and c at 0; agent 2 values only c , at 1. Consider $X_1 = \{b\}$ and $X_2 = \{a, c\}$. Under EFX, agent 1 may remove only a from X_2 , since c is worthless to it; what remains, $\{c\}$, is worth $0 \leq 2$. Agent 2 holds c and faces the single good b , which it values at 0. So X is EFX. It is not EFX₀: removing c from X_2 leaves $\{a\}$, worth $3 > 2$ to agent 1. The allocation $X_1 = \{a\}$, $X_2 = \{b, c\}$ is EFX₀.

The rule holds because removing a good h with $v_i(h) = 0$ leaves $v_i(X_j \setminus \{h\}) = v_i(X_j)$. When every agent values only a few goods, most bundles contain goods that a given agent values at zero, and EFX₀ is close to plain envy-freeness towards every bundle of two or more goods. Two consequences run through the whole paper:

- a good that nobody values cannot simply be handed to an arbitrary agent, because it turns every constraint towards that agent's bundle into envy-freeness; and

- a good that agent j values but agent i does not cannot join a bundle that i envies.

1.2 The question and the answer

Write $R_i = \{g \in M : v_i(g) > 0\}$ for the goods agent i values, its *relevant* goods. An instance is *k-relevant* if $|R_i| \leq k$ for every agent i . For $k \leq 2$, serial dictatorship works: the agents, in any order, each take a favourite remaining good, and the last agent takes everything left. The result is EFX₀ (this is machine-checked in [10]; we reprove it as Corollary 2, which is machine-checked here too, for every order and every choice of favourite). At $k = 3$ serial dictatorship breaks. An agent that values a, b, c at 4, 3, 2 may take a and then see b and c end up in the last agent's bundle together with a third good; removing that third good, which it values at zero, leaves b and c , worth $5 > 4$.

This paper answers the question for $k = 3$.

Theorem 1 (existence). *Every instance with $n \geq 1$ agents and non-negative real additive valuations in which every agent positively values at most three goods has a complete EFX₀ allocation.*

Call an agent that values exactly three goods, ranked $a \succ b \succ c$ by value, *balanced* if it values a at most as much as b and c together.

Theorem 2 (one large bundle). *Every instance with $n \geq 1$ agents and nonnegative real additive valuations in which every agent values exactly three goods and is balanced has a complete EFX₀ allocation in which all but at most one agent receive at most two goods.*

The third result is an algorithm. We measure its running time in an explicit *unit-cost model*, where each list step, comparison of agents or goods, read of an input value or table entry, and comparison or addition of natural numbers of any size costs one operation (Section 6 states the model precisely; Corollary 3, written, not formalized, counts bit operations instead).

Theorem 3 (algorithm). *Given $n \geq 1$ agents, m goods and natural-number values in binary, algorithm *K3ALG* (Section 4) performs at most $400(n+m+1)^4$ operations in the unit-cost model. If every agent positively values at most three goods, it returns a complete EFX₀ allocation.*

Corollary 1 (rational values). *Let every agent positively value at most three goods, with nonnegative rational values. Scale each agent’s values to natural numbers, multiplying them by a common denominator; this preserves every comparison between sums of that agent’s values. Then K3ALG on the scaled values returns a complete allocation that is EFX_0 for the rational values.*

The proofs of Theorems 1–3 are written in full in this paper. They are also checked in Lean 4 (Section 7): Theorem 3 for natural-number values, and Theorems 1 and 2 for values in any linearly ordered cancellative commutative monoid; so are Corollary 1, for nonnegative rationals, and Corollary 4, K3ALG on real values in a comparison model, for any such monoid. Instantiating these at $\mathbb{R}_{\geq 0}$ uses the standard fact that $\mathbb{R}_{\geq 0}$ is such a monoid, which is not a Lean theorem, since core Lean has no real numbers.

Kinds of evidence. Every claim of this paper carries exactly one of five labels, and Table 1 lists the claims with their labels.

- *Lean-checked theorem*: its proof is written in this paper (for the operation bounds of Theorems 3 and 9, as a step-by-step outline in Appendix B), and its statement is checked in Lean, over the trusted base of Section 7.2.
- *Theorem (proof in this paper)*: the proof is written in this paper; it is not formalized.
- *Written, not formalized*: a written argument that relies on material outside this paper’s definitions, namely the text of a program (for counts of its operations and what they compute) or a proof recorded in the repository; it is not formalized.
- *Verified by exhaustive computation*: every case is checked by programs, and the stored certificates are re-checked by an independently written checker.
- *Measurement*: observed in runs of implementations; evidence, not a guarantee.

Every numbered lemma, theorem, proposition and corollary of this paper is a Lean-checked theorem unless its heading gives another label. Lean checks these statements for natural-number values, except Theorems 1 and 2, Lemma 6 and Corollary 4, which it checks for any linearly ordered cancellative commutative monoid, as just explained; Corollary 1, which it checks for nonnegative rationals; and Proposition 2, which concerns the rankings alone. The written proofs apply verbatim to real values. Appendix D.5 names the Lean declarations.

Table 1. The claims of this paper and their evidence (Section 1.2). *Lean-checked for values in any linearly ordered cancellative commutative monoid; that $\mathbb{R}_{\geq 0}$ is such a monoid is a standard fact, not a Lean theorem (core Lean has no real numbers). *Rankings*: the claim involves only the agents’ rankings, whatever the values.

Claim	Values	Evidence
Thm. 1: existence	$\mathbb{R}_{\geq 0}$	Lean-checked theorem*
Thm. 2: one large bundle	$\mathbb{R}_{\geq 0}$	Lean-checked theorem*
Thm. 3: K3ALG is correct	$\mathbb{N}$	Lean-checked theorem
Thm. 3: $400(n + m + 1)^4$ operations, unit cost	$\mathbb{N}$	Lean-checked theorem
Thm. 9: $270(n^4 + n^2m)$ operations, unit cost	$\mathbb{N}$	Lean-checked theorem
Sects. 3, 5: the other lemmas and theorems	$\mathbb{N}$	Lean-checked theorem
Cor. 2: serial dictatorship, two goods	$\mathbb{N}$	Lean-checked theorem
Prop. 2: size of the large bundle	rankings	Lean-checked theorem
Sect. 5.5: r lies in the last block	rankings	Lean-checked theorem
Cor. 1: rational values	$\mathbb{Q}_{\geq 0}$	Lean-checked theorem
Cor. 4: real values, comparison oracle	$\mathbb{R}_{\geq 0}$	Lean-checked theorem*
Lem. 1, 2: threats, safety	$\mathbb{R}_{\geq 0}$	theorem (proof in this paper)
Prop. 4: limits of the shape	$\mathbb{R}_{\geq 0}$	theorem (proof in this paper)
Cor. 3: bit complexity	$\mathbb{N}$	written, not formalized
Sect. 6.5: Python, $O((n + m) \log n)$ after the input	$\mathbb{N}$	written, not formalized
Sect. 8: existence for $n \leq 7$	$\mathbb{R}_{\geq 0}$	verified by exhaustive computation
App. F: Thm. 2, cores with $n \leq 6$, connected $n = 7$	$\mathbb{R}_{\geq 0}$	verified by exhaustive computation
Sect. 6.5: speed of the Python implementation	$\mathbb{N}$	measurement
App. F: tests of LB^+ and K3ALG	$\mathbb{N}$	measurement

Why this is not a corollary of known results. A 3-relevant instance without a complete EFX_0 allocation would refute the additive EFX conjecture itself. Indeed, adding a small $\varepsilon > 0$ to every value gives an instance with positive values, in which EFX and EFX_0 coincide. If each of these perturbed instances had an EFX allocation, one allocation would serve arbitrarily small ε , since there are finitely many allocations, and it would be EFX_0 for the original values, since the inequalities are closed (compare the perturbation lemma of [5]). But the perturbation makes every good relevant to every agent, so it destroys the structure we exploit, and no published result that we know of implies Theorem 1 [9]. We prove it, with a polynomial-time algorithm whose correctness and operation bound, in an explicit unit-cost model, are machine-checked.

1.3 Related work

For additive valuations, complete EFX allocations are known to exist only in special cases, two of them in recent preprints [3,1]: for three agents [5]; when $m \leq n + 3$ [12]; for four agents and at most nine goods [3]; when every good is valued by at most two agents (graphs and multigraphs of agents, with goods as edges) [1,6,13,7]; and for hypergraphs of girth at least four [8]. All of these use the all-goods form EFX_0 ; in the graph and hypergraph results every good is worth zero to the agents it is not incident to, and for results that bound n and m positive values suffice by the perturbation argument above. For monotone valuations that are not additive, EFX allocations need not exist [2,11]. Table 2 compares these results with ours.

We restrict instead how many goods each agent values. For $k \leq 2$, serial dictatorship suffices and has been formalized in Lean [10]. For $k \leq 4$, Viswanathan and Mehta [14] announce the existence of ordinary EFX allocations, with a polynomial-time algorithm and a proof sketch in an extended abstract. Their first phase, which gives a first choice, by a maximum matching, to as many as possible of the agents that value their top good more than their second and third together, resembles our rule R1 (Section 3.4). Their algorithm ends by giving the goods that nobody values to an arbitrary agent, which is harmless for EFX but can break EFX_0 , as Example 1 illustrates. Goods that nobody values are in fact easy to place (Lemma 5); the difficulty under EFX_0 comes from goods that one agent values and another does not, which their EFX argument does not need to address. Since EFX_0 implies EFX, Theorem 1 proves the case $k = 3$ of their result in full.

Table 2. Existence results for complete allocations under restrictions, from the reading notes in `proofs/citations.md` of [9]. *Version cited:* published version if Crossref or the paper confirms one, else arXiv. ^aGoods nobody values go to an arbitrary agent. ^bIncludes additive. ^cExtended abstract with a proof sketch. ^dAn AAMAS 2025 extended abstract has the other two cases.

Result	Restriction (agents, goods, who values what)	Notion	Complete	Valuations	Algorithm	Version cited
[5]	$n = 3$	EFX ₀	yes	additive	pseudo-polynomial	JACM 2024
[12]	$m \leq n + 3$	EFX ₀	yes	monotone	none stated	MOR 2024
[3]	$n = 4, m \leq 9$	EFX ₀	yes	additive	none stated	arXiv
[6]	graphs: at most two valuers per good, two agents share at most one good	EFX ₀	yes	monotone	polynomial time	MOR 2026 (EC 2023)
[1]	multigraphs: at most two valuers per good	EFX ₀	yes	cancelable ^b	polynomial time	arXiv
[13]	multigraphs: bipartite, or at most $\lceil n/4 \rceil - 1$ neighbours per agent, or girth at least six (non-parallel edges)	EFX ₀	yes	monotone	not efficient (stated)	arXiv ^d
[7]	at most two valuers per good, $v_i(g) \in \{0, v_g\}$	EFX ₀	yes	restricted additive	no time bound stated	WINE 2024
[8]	hypergraphs of girth at least four (a good joins its valuers)	EFX ₀	yes	monotone	polynomial time	AAAI 2026
[14]	at most four relevant goods per agent	EFX	yes ^a	additive	polynomial time, announced	AAMAS 2024 ^c
[10]	at most two relevant goods per agent	EFX ₀	yes	additive; monotone	serial dictatorship; Lean-checked	GitHub, v1.2.0
this paper	at most three relevant goods per agent; any number of valuers per good	EFX ₀	yes	additive	$400(n + m + 1)^4$ unit-cost steps; Lean-checked	this paper

Where the difficulty lies. Under ordinary EFX, a good that agent i values at zero never constrains i , so goods valued by some agents and not others are harmless; this is why [14] can give the goods nobody values to an arbitrary agent, and on multigraphs ordinary EFX is easy, so the difficulty there is EFX₀ [13]. Under EFX₀ with complete allocations, every such good must go into some bundle, and it turns the constraint of each agent that does not value it into envy-freeness towards that bundle (the rule of Section 1.1). *The difficulty of Theorem 1 is exactly this combination: goods that some agents value and others do not, together with completeness.* The structural results of Table 2 cope with it when each good has at most two valuers or the valuers form a hypergraph of girth at least four; the others bound n or m . Theorem 1 bounds only the number of goods each agent values: a good may have any number of valuers, and any number of agents to whom it is worthless.

1.4 How to read this paper

Section 2 explains the approach in plain words on one instance, with no formal definitions. Section 3 gives the model and four basic lemmas: the ordinal description of when an agent is safe (Lemma 2), on which the whole design rests; peeling; goods valued by nobody; and the reduction of real values to natural numbers. Section 4 defines the algorithm K3ALG one notion at a time, lists it as pseudocode, and traces it on two examples, one with an owner and one with a rotation. Section 5 proves it correct, with every case written out. Section 6 bounds its running time, in three separate layers. Section 7 describes the Lean formalization: what it covers, what must be trusted, how to read its statements, and what it leaves out. Section 8 explains what the exhaustive computations for small instances establish and shows, by counterexamples, that the shape of Theorem 2 cannot be improved in the obvious ways. Section 9 concludes. The appendices hold the full proof of Lemma 6 and material for checking the artifacts rather than for reading: the cost of each step, the verification of the examples, the repository’s audit of the formalization, a table that maps every object of Section 4 to its Lean definition, and the run totals, certificates and reproducibility details of the computations.

The paper is self-contained. Its claims are also recorded, each with its evidence, in the ledger of the public repository [9]. Every example in this paper was checked by running the repository’s implementations of K3ALG with a brute-force check of the EFX₀ definition, and by the Lean program itself (Appendix C).

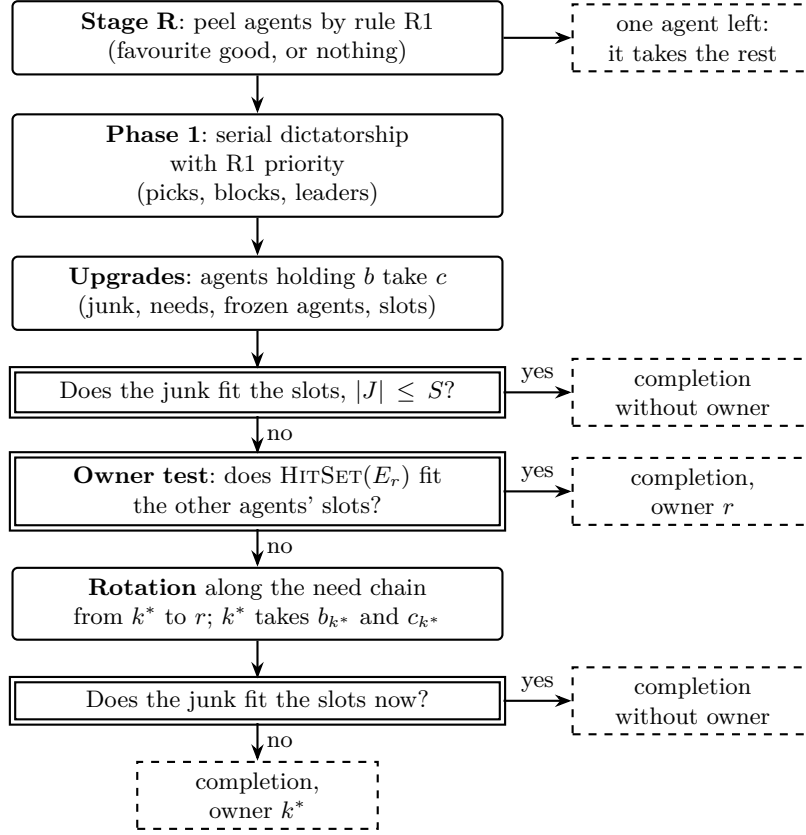


Fig. 1. The stages of K3ALG. Dashed boxes are outputs; double-bordered boxes are tests. Stage R ends when rule R1 applies to no agent (then Phase 1 starts) or when at most one agent or no good is left (then the algorithm returns). Every output is a completion of the current pre-allocation; the owner, if any, receives the only bundle that may have more than two goods.

2 Overview of the Approach

This section explains the algorithm in plain words, on one instance, before any formal definition. Figure 1 shows its stages. Section 4 gives the precise definitions and traces the same instance step by step (Example 2); Section 5 proves everything claimed here.

The instance. Five agents share nine goods $g_0, \dots, g_8$. Each agent's positive values are, from the most to the least valued good:

agent 0: $g_2 \mapsto 6, g_4 \mapsto 5, g_6 \mapsto 2$ agent 3: $g_2 \mapsto 9, g_8 \mapsto 8, g_1 \mapsto 7$
 agent 1: $g_6 \mapsto 5, g_7 \mapsto 4, g_0 \mapsto 3$ agent 4: $g_5 \mapsto 6$
 agent 2: $g_2 \mapsto 9, g_1 \mapsto 7, g_3 \mapsto 5$

Every other value is 0. Note that g_2 is the favourite good of three agents.

Step 1: peeling. Agent 4 values only g_5 . If it takes $\{g_5\}$ and leaves, then whatever the others do afterwards, agent 4 values every other bundle at 0, and nobody can strongly envy a bundle of one good: removing that good leaves nothing. More generally, an agent whose favourite remaining good p is worth at least as much as all its other remaining goods together may take $\{p\}$ and leave (*rule R1*, Lemma 3); an agent that values none of the remaining goods may leave with nothing. We repeat this while it applies. In the example only agent 4 leaves: agent 0, for instance, values its favourite g_2 at $6 < 5 + 2$.

When peeling stops, every remaining agent values exactly three of the remaining goods, $a \succ b \succ c$, and is *strictly balanced*: $v(a) < v(b) + v(c)$ (Lemma 4). An agent valuing at most two goods, or with $v(a) \geq v(b) + v(c)$, would still be peeled.

Step 2: a serial dictatorship with R1 priority. The remaining agents now take turns, and each takes its favourite good among those still available: its *pick*. The order matters. If some waiting agent has already lost one of its goods, it goes next. Such an agent has at most two of its goods left, so rule R1 applies to it in what remains, and taking its favourite good mimics peeling. Only when every waiting agent still has all three of its goods does a new agent, a *leader*, take its top good a . A leader and the agents served after it, up to the next leader, form a *block*.

In the example all four agents start with all their goods, so agent 0 (the first in index order) leads block 1 and picks g_2 . Agents 2 and 3 have lost g_2 : agent 2 picks its $b = g_1$, and then agent 3, which has lost g_2 and g_1 , picks its $b = g_8$. The only agent still waiting, agent 1, has all its goods; it leads block 2 and picks its top, g_6 . Four goods are left over: g_0, g_3, g_4, g_7 .

Step 3: why the leftovers are the whole difficulty. If no good were left over, every agent would hold at most one good, and the allocation would be EFX_0 for the trivial reason that nobody can strongly envy a bundle of at most one good. The leftovers, which we call *junk*, must go somewhere, and in general they create bundles of two or more goods, which can be strongly envied. Here the two notions part ways. Under ordinary EFX ,

adding to a bundle a good that agent i does not value never creates a new constraint for i , since only goods that i values may be removed. Under EFX₀, it turns i 's constraint towards that bundle into envy-freeness, as in Example 1. Lemma 2 says exactly when a strictly balanced agent with $v(a) > v(b) > v(c)$ is safe:

- holding a , it is safe unless b and c lie together in a bundle of three or more goods;
- holding b and c , it is always safe;
- holding only b of its goods, it is safe if and only if a is *alone* (the only good of its bundle);
- holding only c , it is safe if and only if a and b are alone;
- holding none of its goods, it is safe if and only if a , b and c are all alone.

So an agent that lost goods to earlier agents is safe as long as those goods stay alone. We call them its *needs*. In the example, agent 2 holds its b and needs g_2 alone, and agent 3 holds its b and needs g_2 alone too. So g_2 , agent 0's pick, must stay alone: agent 0 may receive no junk. Such an agent is *frozen*.

Step 4: slots and upgrades. An agent whose pick nobody needs, a *terminal*, may receive one junk good (a *slot*), and an agent without a pick may receive two. An agent that picked its b and whose c is junk may also take its c (an *upgrade*): holding b and c is always safe, and the agent then needs nothing. The upgrade is allowed only if nobody needs its b alone, since b will now share a bundle. In the example, agent 2 holds $b = g_1$, its $c = g_3$ is junk, and nobody needs g_1 , so agent 2 takes g_3 . Agent 3 cannot upgrade: its $c = g_1$ belongs to agent 2. Now agent 0 is frozen (agent 3 still needs g_2), agents 1 and 3 are terminals with one slot each, and the junk is g_0, g_4, g_7 : three goods for two slots.

Step 5: the owner. When the junk exceeds the slots, one agent, the *owner*, takes all the junk that does not fit, so its bundle has three or more goods. (Usually every other slot is then filled; when the list of protecting goods that K3ALG builds repeats a good, one slot can stay empty and the owner's bundle is larger, Remark 1.) By Lemma 2, and because goods needed alone never go to the owner, such a bundle can hurt only an agent x that holds its top a_x and would find both b_x and c_x in it. Such an agent is *exposed*, and it is protected if one of b_x, c_x goes into a slot instead. The natural candidate owner is r , the last agent served that is not upgraded; here $r = 1$. Agent 0 is exposed for r : it holds its top g_2 , its $b = g_4$ is junk,

and its $c = g_6$ is r 's own pick. So g_4 goes into agent 3's slot and the owner takes g_0 and g_7 :

$$X_0 = \{g_2\}, X_1 = \{g_6, g_0, g_7\}, X_2 = \{g_1, g_3\}, X_3 = \{g_8, g_4\}, X_4 = \{g_5\}.$$

This allocation is EFX_0 ; Table 6 checks it agent by agent. Filling agent 3's slot with g_0 instead would give the owner $\{g_6, g_4, g_7\}$. Agent 0 values that bundle at $7 > 6$ and still at 7 after removing g_7 , which it values at zero, so the allocation would not be EFX_0 . It would still be EFX , since removing g_4 or g_6 leaves 2 or 5: here, again, the difference between the two notions is a good worthless to the envious agent.

Why does r usually work? An exposed agent took its top while b and c were still available, so it found all three of its goods available: it is a leader. So different exposed agents lead different blocks. Each of them has a slot at its disposal inside its own block: its own, if it is a terminal; if it is frozen, the slot of the terminal at the end of a chain of needs inside the block (a *need chain*). In the example, agent 3 needs agent 0's pick and is a terminal, so the need chain is $0 \rightarrow 3$, and agent 3's slot protects agent 0. So there is one slot per exposed agent, except possibly for the exposed agent in r 's own block, because r 's own slot is not available to others.

Step 6: the one rotation. The count above fails in exactly one situation, the *bad case* (Theorem 5): the leader k^* of r 's block is exposed and frozen, every need chain from k^* ends at r , and no two exposed agents can be protected by the same junk good. Then we rotate along the need chain from k^* to r : every agent of the chain takes the good it needed from its predecessor, and k^* gives up its top a and takes b and c instead (an upgrade). Every agent of the chain now holds a good it ranks above its old pick, so the goods that must stay alone can only become fewer, and k^* can now own the large bundle (Theorem 6). Example 3 in Section 4 shows a rotation, and Figure 2 draws it.

What is proved. The allocation that K3ALG returns is EFX_0 whenever every agent values at most three goods (Theorem 3), and at most one agent of Stage L, the owner, receives more than two goods. The proof in Section 5 works for every order of the dictatorship with R1 priority and every order of the upgrades, so K3ALG's choice of the smallest index is only one way to break ties.

3 Preliminaries

3.1 Model and notation

An *instance* consists of a finite set N of $n \geq 1$ agents, a finite set M of m goods, and nonnegative additive valuations v_i , $i \in N$. Agents and goods are numbered, and “index order” and “the first” refer to these numbers; in examples, agents are $0, 1, 2, \dots$ and goods are $g_0, g_1, \dots$. The relevant goods of agent i are $R_i = \{g \in M : v_i(g) > 0\}$. An allocation X is *EFX₀* if every agent is *safe* in it, where agent i is safe in X if

$$v_i(X_i) \geq v_i(X_j \setminus \{h\}) \quad \text{for every agent } j \neq i \text{ and every good } h \in X_j.$$

It is convenient to name the largest right-hand side. For a nonempty set B of goods, the *threat* of B to agent i is

$$\theta_i(B) = \max_{h \in B} v_i(B \setminus \{h\}) = v_i(B) - \min_{h \in B} v_i(h),$$

and $\theta_i(\emptyset) = 0$. Agent i is safe in X if and only if $v_i(X_i) \geq \theta_i(X_j)$ for every $j \neq i$. A good is *alone* in X if it is the only good of its bundle.

Lemma 1 (threats; theorem, proof in this paper). *Let B be a set of goods and i an agent. (a) If $|B| \leq 1$, then $\theta_i(B) = 0$: a bundle of at most one good is never strongly envied. (b) If B contains a good outside R_i , then $\theta_i(B) = v_i(B)$. (c) In every case $\theta_i(B) \leq v_i(B)$, and $\theta_i(B)$ depends only on the values $v_i(g)$, $g \in B$.*

Proof. By additivity, $v_i(B \setminus \{h\}) = v_i(B) - v_i(h)$ is largest when $v_i(h)$ is smallest. (a) If $B = \{h\}$, removing h leaves nothing. (b) The smallest value in B is then 0. (c) Values are nonnegative. $\square$

Part (b) is the mechanism of Example 1: towards a bundle that contains a good it does not value, an agent needs full envy-freeness.

3.2 Rankings and balance

When $|R_i| = 3$ we name agent i ’s relevant goods a_i, b_i, c_i so that $v_i(a_i) \geq v_i(b_i) \geq v_i(c_i) > 0$, breaking ties between equal values by index (the good with the smaller index comes first). This gives a strict *ranking* $a_i \succ_i b_i \succ_i c_i$, fixed once and for all; we write $g \succ_i h$ when i ranks g above h . Agent i is

- *balanced* if $v_i(a_i) \leq v_i(b_i) + v_i(c_i)$, that is, if no relevant good is worth more than the other two together (equivalently, $2v_i(g) \leq v_i(M)$ for every good g , the form used in Lean);
- *strictly balanced* if $v_i(a_i) < v_i(b_i) + v_i(c_i)$;
- *top-heavy* if $v_i(a_i) \geq v_i(b_i) + v_i(c_i)$.

Much of the paper depends on the values only through the rankings. Given rankings $a_i \succ_i b_i \succ_i c_i$ of three goods per agent, a valuation is *consistent* with them if $v_i(a_i) \geq v_i(b_i) \geq v_i(c_i) > 0$ and $v_i(g) = 0$ for every other good, for every agent i .

3.3 When is an agent safe? The ordinal lemma

The next lemma describes exactly when a strictly balanced agent with three relevant goods is safe. It is the key to the design of the algorithm: every notion of Section 4 comes from one of its cases.

Lemma 2 (cases of safety; theorem, proof in this paper). *Let agent i have $R_i = \{a, b, c\}$ with $v_i(a) > v_i(b) > v_i(c) > 0$ and $v_i(a) < v_i(b) + v_i(c)$, and let X be an allocation.*

- (i) *The eight sums of subsets of $\{a, b, c\}$ are ordered as $0 < c < b < a < b + c < a + c < a + b < a + b + c$, writing x for $v_i(x)$.*
- (ii) *Agent i is safe in X if and only if one of the following holds:*
 - (T) *i holds a , and either i also holds b or c , or b and c are not together in a bundle of at least three goods;*
 - (P) *i holds b and c ;*
 - (B) *i holds b , and a is alone;*
 - (C) *i holds c , and a and b are alone;*
 - (E) *a , b and c are all alone.*
- (iii) *Suppose instead only $v_i(a) \geq v_i(b) \geq v_i(c) > 0$ and $v_i(a) < v_i(b) + v_i(c)$ (ties allowed), with a, b, c labelled consistently with the values. Then each of (T), (P), (B), (C), (E) still implies that i is safe.*

Proof. (i) $c < b < a$ holds by assumption; $a < b + c$ is strict balance; $b + c < a + c$ since $b < a$; $a + c < a + b$ since $c < b$; and $a + b < a + b + c$ since $c > 0$.

(ii) Let $S = X_i \cap R_i$ be the relevant goods that i holds, so $v_i(X_i) = v_i(S)$. For another bundle X_j , let $U = X_j \cap R_i$. By Lemma 1, $\theta_i(X_j) = 0$ if $U = \emptyset$ or $|X_j| = 1$; $\theta_i(X_j) = v_i(U)$ if X_j has a good outside R_i ; and if $X_j = U$ has two or three goods, $\theta_i(X_j)$ is $v_i(U)$ minus its smallest value: the larger of the two values if $|U| = 2$, and $a + b$ if $U = \{a, b, c\}$. We go through all eight possible sets S .

- $S \supseteq \{a, b\}$ or $S \supseteq \{a, c\}$. Then $v_i(S) \geq a + c$, and every other bundle meets R_i in at most one good, so its threat is at most $b < a + c$. Safe; this is (T).
- $S = \{b, c\}$. Only a lies outside S , so every threat is at most $a < b + c = v_i(S)$. Safe; this is (P).
- $S = \{a\}$. If b and c lie in different bundles, each threat is at most $b < a$. If they lie in the same bundle X_j and $X_j = \{b, c\}$, the threat is $b < a$. If $X_j \supsetneq \{b, c\}$, then X_j contains a good outside R_i (the good a is i 's), so $|X_j| \geq 3$ and the threat is $b + c > a$. So i is safe if and only if b and c are not together in a bundle of at least three goods: this is (T).
- $S = \{b\}$. If a is not alone, its bundle is $\{a, c\}$, with threat a , or contains a good outside R_i , with threat at least a ; either way the threat exceeds b and i is not safe. If a is alone, its threat is 0, and the bundle of c has threat at most $c < b$. So i is safe if and only if a is alone: this is (B).
- $S = \{c\}$. As before, a must be alone. If b is not alone, its bundle is $\{a, b\}$ (threat a) or contains a good outside R_i (threat at least b); either way the threat exceeds c . If a and b are both alone, every threat is 0. So i is safe if and only if a and b are alone: this is (C).
- $S = \emptyset$. Then $v_i(S) = 0$, and a bundle with a good of R_i and at least two goods has a positive threat. So i is safe if and only if a , b and c are all alone: this is (E).

Conversely, each of (T), (P), (B), (C), (E) describes one of the safe situations above: (T) is the first and third bullets; (P) is $S = \{b, c\}$ or $S = \{a, b, c\}$; (B) forces $a \notin S$ (as a is alone and not i 's), so $S = \{b\}$ with a alone, or $S = \{b, c\}$; (C) forces $S = \{c\}$ with a and b alone; and (E) means that S is empty or a single alone good, which is (T), (B) or (C).

(iii) The “safe” directions above use only $a \geq b \geq c > 0$ and $a < b + c$. Under (T), if i also holds b or c , then $v_i(S) \geq a + c$ while every other bundle meets R_i in one good, worth at most $b \leq a + c$; if i holds only a , the goods b and c are in different bundles or form exactly the bundle $\{b, c\}$, so every threat is at most $b \leq a$. Under (P) every threat is at most $a < b + c$. Under (B) the only bundle meeting R_i outside X_i besides $\{a\}$ has threat at most $c \leq b$. Under (C) and (E) every threat is 0. $\square$

How the lemma is used. First, it explains the design. An agent holding only b (case B), only c (case C) or nothing (case E) is safe exactly when the goods it ranks above its own stay alone; these goods will be its *needs*. Case (P) is the *upgrade*. Case (T) fails only if b and c end up together in a bundle of three or more goods, which only the owner's bundle can be; this is the

owner constraint. Second, it shows that, among strictly balanced agents with three relevant goods and strict rankings, EFX_0 depends only on the rankings, which the computations of Section 8 use; by (iii), certifying the strict rankings covers tied values too. The soundness theorem of Section 5 does not cite the lemma: it re-proves the half it needs directly from the values, so that it covers ties and balanced agents with $v_i(a_i) = v_i(b_i) + v_i(c_i)$.

3.4 Peeling

Peeling removes one agent, with a bundle, in a way that can never hurt the rest.

Lemma 3 (peeling). *Let $i \in N$ and $P \subseteq M$ satisfy*

- (i) $v_i(P) \geq v_i(M \setminus P)$, and
- (ii) $|P| \leq 1$, or every good of P is worthless to every agent other than i .

If the instance with agents $N \setminus \{i\}$ and goods $M \setminus P$ has an EFX_0 allocation Y , then Y together with $X_i = P$ is an EFX_0 allocation of the whole instance.

Proof. Agent i : every other bundle Y_j lies in $M \setminus P$, so $\theta_i(Y_j) \leq v_i(Y_j) \leq v_i(M \setminus P) \leq v_i(P)$ by Lemma 1(c) and (i). An agent $j \neq i$ towards P : if $|P| \leq 1$, then $\theta_j(P) = 0$ by Lemma 1(a); otherwise $v_j(P) = 0$ by (ii), so $\theta_j(P) = 0$. An agent $j \neq i$ towards Y_k , $k \neq i, j$: this constraint is one of Y 's, and j still holds Y_j . $\square$

Rule R1 is the case $|P| \leq 1$ used by K3ALG. Let p be agent i 's favourite good among the remaining goods M (the first good of largest value $v_i(p)$). If $v_i(p) = 0$, agent i values no remaining good and leaves with $P = \emptyset$; then (i) reads $0 \geq 0$. If $v_i(p) > 0$ and $v_i(M \setminus \{p\}) \leq v_i(p)$, agent i leaves with $P = \{p\}$. Rule R1 applies, for instance, to every agent that values at most two remaining goods, and to every top-heavy agent. As a first consequence, serial dictatorship works for two relevant goods.

Corollary 2 (two relevant goods). *If $|R_i| \leq 2$ for every agent, then serial dictatorship, in any order of the agents, each taking a favourite remaining good (any good of largest value to it among those left, or nothing once no good is left), with the last agent taking all remaining goods, returns an EFX_0 allocation.*

Proof. By induction on the number of agents. One agent faces no constraint. Otherwise the first agent i has at most two relevant goods among the remaining ones, and takes a favourite remaining good p ; its other relevant good, if any, is worth at most $v_i(p)$, so $v_i(M \setminus \{p\}) \leq v_i(p)$. The rest of the procedure is serial dictatorship on the smaller instance, which is EFX₀ by induction, and Lemma 3 with $P = \{p\}$ finishes the proof; if no good is left, i takes $P = \emptyset$, and (i) reads $0 \geq 0$. $\square$

In Lean, for natural-number values: `EFX.Inst.sdRun_efx0` (every run: any order of all agents, any choice of favourite at each step) and `EFX.Inst.serialDict_efx0` (a computable run, taking the first favourite, in any order); the existence form is `EFX.exists_efx0_of_count`. The form for values in an ordered type is not formalized. When rule R1 applies to no agent, every agent has exactly the structure that the rest of the algorithm needs.

Lemma 4 (when R1 applies to nobody). *Let agent i value at most three of the goods M , and suppose that rule R1 does not apply to i . Then i values exactly three goods of M , is strictly balanced, and $2v_i(g) < v_i(M)$ for every good $g \in M$.*

Proof. Rule R1 fails, so $v_i(p) > 0$ and $v_i(M \setminus \{p\}) > v_i(p)$ for the favourite p . If i valued at most two goods of M , then $v_i(M \setminus \{p\})$ would be the value of at most one good, which is at most $v_i(p)$. So i values exactly three goods, p is its top a_i , and $v_i(b_i) + v_i(c_i) = v_i(M \setminus \{p\}) > v_i(a_i)$. Finally $2v_i(g) \leq 2v_i(p) < v_i(p) + v_i(M \setminus \{p\}) = v_i(M)$. $\square$

3.5 Goods that nobody values

K3ALG does not need the next lemma, because its Stage L tolerates goods that nobody values. We include it because it shows that such goods are easy to place (Section 1.3) and because the reduction to cores in Section 8 uses it. The *envy graph* of an allocation Y has an arc $i \rightarrow j$ whenever $v_i(Y_j) > v_i(Y_i)$.

Lemma 5 (goods that nobody values). *Let $Z \neq \emptyset$ be the set of goods that no agent values. If the instance without Z has an EFX₀ allocation, so does the instance.*

Proof. Let Y be an EFX₀ allocation of $M \setminus Z$. While the envy graph of Y has a cycle $i_1 \rightarrow i_2 \rightarrow \dots \rightarrow i_k \rightarrow i_1$, give each agent of the cycle the bundle of its successor. The new allocation has the same bundles, and no

agent's value decreases, strictly increasing for the agents on the cycle. It is still EFX_0 : an agent now faces the same bundles as before, except possibly its own old bundle Y_i , whose threat is at most $v_i(Y_i)$ by Lemma 1(c), and its own value did not decrease. The sum of all agents' values strictly increases and only finitely many reassignments of the same bundles exist, so the loop stops, with an acyclic envy graph. An acyclic finite digraph has a source s , an agent that nobody envies: $v_i(Y_s) \leq v_i(Y_i)$ for all i . Let $X_s = Y_s \cup Z$ and $X_i = Y_i$ otherwise. For $i \neq s$, $\theta_i(X_s) \leq v_i(X_s) = v_i(Y_s) \leq v_i(Y_i)$, since $v_i(Z) = 0$; all other constraints are unchanged. $\square$

3.6 Real values reduce to natural numbers

The Lean model has natural-number values. The next lemma transfers results from natural numbers to real values, and also lets the algorithm run on real inputs (Section 6.4).

Lemma 6 (real values reduce to natural numbers). *Let v be non-negative real additive valuations with $|R_i| \leq 3$ for every agent i . There are natural-number additive valuations w with the same relevant goods such that, for every agent i and all sets $S, T \subseteq M$,*

$$v_i(S) \leq v_i(T) \iff w_i(S) \leq w_i(T).$$

Consequently an allocation is EFX_0 for v if and only if it is EFX_0 for w , and an agent values exactly three goods and is balanced under v if and only if it does under w .

Proof (sketch; full proof in Appendix A). Fix i . Goods outside R_i contribute zero to both sides, and goods common to S and T can be cancelled, so only comparisons between disjoint subsets of the at most three relevant goods matter. With values $a \geq b \geq c > 0$, those comparisons are decided by the ties among a, b, c and by the sign of $a - (b + c)$, because $b < a + c$ and $c < a + b$ always hold. So it suffices to pick one natural-number triple for each of the finitely many patterns, such as $(4, 3, 2)$ for $a > b > c$ with $a < b + c$. EFX_0 and balance are conjunctions of such comparisons. $\square$

The written proofs of Theorems 1 and 2 below work verbatim for real values. Lemma 6 is what carries the Lean proofs, which are about natural numbers, over to any linearly ordered cancellative commutative monoid, and so, by the standard fact that $\mathbb{R}_{\geq 0}$ is one, to real values (Section 7); in Lean it is `EFX.112`, with `EFX.efx0_iff_of_agree` for the transfer of EFX_0 .

4 The Algorithm K3ALG

K3ALG has two stages. *Stage R* peels agents by rule R1. *Stage L* runs a construction called LB^+ on the agents and goods that remain. This section introduces the notions of Stage L one at a time, each just before it is needed and each with the intuition behind it (Section 2 gave the big picture); Table 4 collects them. Algorithm 1 is the complete pseudocode, and Examples 2 and 3 trace it.

Input. As formalized, K3ALG receives $n \geq 1$, m and the valuation as a function v that gives each agent i and good g a value $v_i(g) \in \mathbb{N}$ (the field $\mathbf{v}$ of the model's instance, Section 7.2). The program reads it as a dense $n \times m$ table, at one unit per read of a value $v_i(g)$ (Section 6.2). Correctness needs every agent to value at most three goods; the operation bound holds for every input. Table 3 summarizes the algorithm and the invariant behind each of its steps.

4.1 Stage R

Let A be the agents still present and M the goods still unallocated; initially all of them. While $|A| \geq 2$ and $M \neq \emptyset$, K3ALG looks for the first agent $i \in A$ to which rule R1 applies on M (Section 3.4), removes it from A , and, if i leaves with a good p , sets $X_i = \{p\}$ and removes p from M . If there is no such agent, Stage R ends and Stage L starts. If only one agent is left, it receives all remaining goods; if no good is left, the remaining agents receive nothing.

By Lemma 3 and induction on the number of agents, the final allocation is EFX₀ as soon as Stage L returns an EFX₀ allocation of what it receives. By Lemma 4, when Stage L starts with 3-relevant input, every agent of A values exactly three goods of M and is strictly balanced. Goods that no agent of A values may remain in M ; Stage L does not need to remove them.

From now on, until the end of Section 5, we consider one Stage-L instance: a set N of agents and a set M of goods in which every agent values exactly three goods, ranked $a_i \succ_i b_i \succ_i c_i$, and is balanced. K3ALG computes the rankings by sorting each agent's three goods by value, larger first, ties by index. Balanced (not necessarily strictly) is all that the correctness proofs need, and goods valued by nobody are allowed. So everything in Sections 4.2–5 applies to the instances of Theorem 2 as well.

Table 3. K3ALG at a glance. Each invariant is proved at the place named in parentheses.

Step	What it does	Invariant
Input	$n \geq 1$ agents, m goods, a table of values $v_i(g) \in \mathbb{N}$. Correctness assumes that every agent values at most three goods; the operation bound does not.	–
Stage R	Peel by rule R1: an agent whose favourite remaining good is worth at least all its other remaining goods takes it and leaves; an agent that values no remaining good leaves with nothing.	a peeled agent is safe, and nobody strongly envies its bundle (Lemma 3); afterwards every agent values exactly three goods and is strictly balanced (Lemma 4)
Phase 1	Serial dictatorship with R1 priority: an agent that has lost a good goes first; otherwise a new leader takes its top. Gives the picks, blocks and leaders.	losses stay inside a block: every good an agent ranks above its pick was picked earlier in its own block ((B2), Lemma 7)
Upgrades	An agent that picked its b , whose c is junk and whose b nobody needs alone, also takes its c .	junk and upgraded goods are not needed alone: the state is valid (Lemma 8)
Completion	Picks to their pickers, c_u to each upgraded u , junk into the slots of the terminals, the rest of the junk to the owner, if any.	all non-owner bundles have at most two goods (Theorem 4)
Owner test	If the junk exceeds the slots: r , the last agent not upgraded, is a valid owner iff $\text{HITSET}(E_r)$ fits the other agents' slots; then complete with owner r , putting $\text{HITSET}(E_r)$ into slots.	owner constraint: no agent that holds its top a_x and is not upgraded finds both b_x and c_x in the owner's bundle (Lemma 9, Proposition 1)
Rotation	Otherwise, rotate the picks along the need chain from k^* to r ; k^* takes b_{k^*} and c_{k^*} . Complete without owner if the junk now fits, else with owner k^* .	NA shrinks ($NA' \subseteq NA$) and k^* becomes upgraded; the state stays valid, and k^* is a valid owner if the junk still exceeds the slots (Theorem 6)
Output	A complete allocation.	EFX_0 when every agent values at most three goods; at most one bundle of Stage L has more than two goods; at most $400(n + m + 1)^4$ and at most $270(n^4 + n^2m)$ unit-cost operations (Theorems 3 and 9)

4.2 Phase 1: a serial dictatorship with R1 priority

Picks. Phase 1 processes the agents one at a time. Let G be the set of goods not yet picked, initially M . At its turn, agent i takes its $\succ_i$ -favourite good of $R_i \cap G$ as its *pick* Y_i , or no pick ($Y_i = \perp$) if none of its goods is left, and Y_i leaves G . *Intuition:* a pick is a bundle of one good, which nobody can strongly envy; everything else is about what the pick costs the agents that wanted it.

R1 priority. The next agent is chosen as follows.

- *R1 step:* if some unprocessed agent has at most two of its goods left in G , the next agent is one of them.
- *Insertion step:* otherwise every unprocessed agent still has all three goods, and the next agent is any unprocessed agent. It picks its top.

K3ALG takes the unprocessed agent with the smallest index in both cases. *Intuition:* an agent with at most two goods left satisfies rule R1 in the remaining instance, so serving it next mimics peeling. Serving agents who lost goods before anyone else takes a new top keeps the damage done by each insertion local, as the invariant (B2) of Section 5.1 makes precise.

Blocks and leaders. A *block* is an insertion step together with the R1 steps that follow it, up to the next insertion step. Its inserted agent is the block's *leader*. The first step is always an insertion step, since initially every agent has all its goods, so every agent belongs to exactly one block, and each block has exactly one leader, its first agent. A leader picks its top. *Intuition:* a block collects the agents served in the wake of one insertion, and every agent loses goods only to earlier agents of its own block (invariant (B2) of Section 5.1).

Leftover goods. The goods never picked in Phase 1 are the *leftover* goods $J_0 = G$ at the end of Phase 1.

4.3 Needs, frozen agents and upgrades

The rest of Stage L changes the picks at most once (the rotation) and gives some agents one extra good each (the upgrades). A pair $P = (Y, U)$ describes the state.

Definition 1 (pre-allocation). A pre-allocation $P = (Y, U)$ consists of

- picks $Y_i \in R_i \cup \{\perp\}$ for all agents i , no good picked twice, and

- a set U of upgraded agents such that every $u \in U$ has $Y_u = b_u$ and also holds c_u ; the goods c_u ($u \in U$) are distinct and are nobody's pick.

Its junk is the set J of goods that are neither a pick nor a good c_u with $u \in U$.

Definition 2 (needs). For an agent $i \notin U$, its needs are the goods it ranks above its pick, $N_i = \{g \in R_i : g \succ_i Y_i\}$, and $N_i = R_i$ if $Y_i = \perp$. The set of goods needed alone is $NA = \bigcup_{i \notin U} N_i$. We say that i needs g when $g \in N_i$.

Intuition: by Lemma 2, an agent holding its b , its c , or nothing is safe as long as its needs are alone (cases B, C, E). Upgraded agents need nothing (case P). An agent holding its top has no needs.

Definition 3 (frozen agents and terminals). An agent $i \notin U$ is frozen if its pick is needed alone, $Y_i \neq \perp$ and $Y_i \in NA$. The agents that are neither upgraded nor frozen are terminals; an agent without a pick is always a terminal.

Intuition: a frozen agent must keep its pick alone, so it receives nothing else. The name “terminal” comes from need chains (Section 4.5), which end at terminals.

Upgrades. Starting from $Y =$ the Phase 1 picks, $U = \emptyset$ and $J = J_0$, the upgrade loop repeats: while some agent $k \notin U$ has

$$Y_k = b_k, \quad c_k \in J, \quad b_k \notin NA \text{ (computed for the current } U),$$

add such a k to U and remove c_k from J . K3ALG upgrades the first such agent each time. *Intuition:* holding b and c is always safe (case P), and the upgraded agent stops needing its top alone. But b_k will now share a bundle with c_k , so the upgrade is allowed only if nobody needs b_k alone. Upgrades only remove sets N_k from the union NA , so NA only shrinks as U grows. When the loop stops:

(UT) no agent $k \notin U$ has $Y_k = b_k$, $c_k \in J$ and $b_k \notin NA$.

Definition 4 (validity). A pre-allocation is valid if (V1) no junk good is needed alone, $J \cap NA = \emptyset$, and (V2) $b_u \notin NA$ and $c_u \notin NA$ for every $u \in U$.

Lemma 8 shows that the state after the upgrades is valid. Validity is what the soundness theorem needs: every good needed alone is then the pick of a frozen agent, which holds it alone.

4.4 Slots, completions and the owner

Definition 5 (slots and bases). *Every agent i has $\text{cap}(i)$ slots: none if it is upgraded or frozen, one if it is a terminal with a pick, and two if it is a terminal without a pick. Let $S = \sum_i \text{cap}(i)$ be the number of slots and $\omega = |J| - S$ the overflow. The base of an agent $i \notin U$ is its pick, $\text{base}(i) = \{Y_i\}$, or $\text{base}(i) = \emptyset$ if it has none; the base of $i \in U$ is $\text{base}(i) = \{b_i, c_i\}$.*

Intuition: every agent other than the owner will hold its base plus at most $\text{cap}(i)$ junk goods, so at most two goods; a terminal's pick is not needed alone, so it may share a bundle.

Definition 6 (completion). *Let P be a pre-allocation. Choose either no owner or an owner o , which must be a terminal or an upgraded agent. A completion of P is an allocation X obtained as follows. Choose pairwise disjoint sets $C_i \subseteq J$ for the terminals $i \neq o$, with $|C_i| \leq \text{cap}(i)$. Give every terminal $i \neq o$ the bundle $\text{base}(i) \cup C_i$, every other agent $i \neq o$ its base, and the owner $\text{base}(o) \cup (J \setminus \bigcup_i C_i)$. Without an owner, the sets C_i must cover J , which is possible exactly when $|J| \leq S$. The completion satisfies the owner constraint (OC) if no agent $x \notin U$, $x \neq o$, with $Y_x = a_x$ has both b_x and c_x in X_o .*

Intuition: the owner's bundle is the only one that may have three or more goods, and by case (T) of Lemma 2 an agent holding its top is hurt by such a bundle only if both its other goods are in it. Without an owner, (OC) holds vacuously.

Definition 7 (exposed agents). *For a candidate owner o , the agents exposed for o are*

$$E_o = \{x \notin U, x \neq o : Y_x = a_x \text{ and } \{b_x, c_x\} \subseteq J \cup \text{base}(o)\},$$

and for $x \in E_o$ we write $\pi_x = \{b_x, c_x\} \cap J$ for the junk part of its pair.

Intuition: these are the agents that the owner's bundle could hurt: their b and c are either junk, which might go to the owner, or already in the owner's base. An exposed agent is protected if a good of π_x goes into some other agent's slot.

Definition 8 (valid owner). *Let P be valid with $\omega \geq 1$. A terminal or upgraded agent o is a valid owner if some set $H \subseteq J$ of junk goods with $|H| \leq S - \text{cap}(o)$ meets $\{b_x, c_x\}$ (equivalently, meets π_x) for every $x \in E_o$.*

Lemma 9 shows that a valid owner has a completion satisfying (OC): put the goods of H into the slots of the agents other than o .

4.5 The candidate owner, need chains and the rotation

The candidate r . Let r be the last agent processed in Phase 1 that is not upgraded. Lemma 10 shows that r exists and is a terminal, and that everyone processed after r is upgraded. *Intuition:* nobody processed after r needs anything alone, so r 's own pick is safe to share, and the agents that r 's bundle could hurt were processed earlier.

The hitting list. For a list E of exposed agents (in index order), K3ALG builds a list $\text{HITSET}(E)$ of junk goods that meets every pair $\{b_x, c_x\}$, $x \in E$. For $z \in E$ let $\text{one}(z)$ be b_z if b_z is junk and c_z otherwise. If two agents $x \neq y$ of E share a junk good $g \in \pi_x \cap \pi_y$ (the first such x in E , then the first such y , then the first such g among b_x, c_x), then $\text{HITSET}(E)$ is g followed by $\text{one}(z)$ for every other $z \in E$ in order; otherwise it is $\text{one}(z)$ for every $z \in E$. So it has $|E| - 1$ entries if two sets π_x meet and $|E|$ entries otherwise; it may list a good twice. Proposition 1 shows that r is a valid owner exactly when $\text{HITSET}(E_r)$ has at most $S - \text{cap}(r)$ entries, so this simple list decides the owner test exactly, without any minimum hitting set.

Need chains. *Intuition:* if x is frozen, some agent needs x 's pick; if that agent is frozen too, some agent needs its pick; and so on. Lemma 12 shows that this always ends at a terminal in x 's block.

Definition 9 (need chain). *A need chain from a frozen agent x is a sequence $x = x_0, x_1, \dots, x_t$ of distinct agents, $t \geq 1$, such that each $x_{s+1} \notin U$ needs Y_{x_s} , the agents $x_0, \dots, x_{t-1}$ are frozen, and x_t is a terminal.*

The rotation. Suppose r is not a valid owner. Let k^* be the first agent of E_r in r 's block; Theorem 5 shows that it exists and is the only exposed agent there, and that every need chain from k^* ends at r . K3ALG builds a need chain from k^* by scanning the agents processed after k^* in order: it appends an agent j whenever the chain's current last agent is frozen, $j \notin U$, and j needs the current last agent's pick. Along the chain $k^* = x_0, x_1, \dots, x_t = r$, the *rotation* makes each x_s ($s \geq 1$) take the pick of x_{s-1} , gives k^* its b_{k^*} as pick and c_{k^*} as upgrade good, and adds k^* to U . The pick of r , if any, becomes junk. *Intuition:* every agent of the chain receives a good it needed, and k^* switches from case (T) to case (P), so it can own the large bundle without being hurt by it.

4.6 Notation

Table 4 is a dictionary of the state of Stage L, for reference while reading the examples and proofs.

4.7 Pseudocode

Algorithm 1 lists K3ALG as it is formalized: the Lean program `EFX.K3.algoC` performs these steps literally, with the same list orders and tie-breaks. In the listing, rule R1 *applies* to $i \in A$ if, for the first good p of M with the largest value $v_i(p)$, either $v_i(p) = 0$ or $v_i(M \setminus \{p\}) \leq v_i(p)$; and $NA(Y, U)$ is the set of goods that some agent of $A \setminus U$ ranks above its pick, or values at all if it has no pick.

Remarks on the listing.

1. The formalized program also says what happens if r (line 22) or k (line 24) does not exist: in the first case it returns the completion without owner, in the second the completion with owner r and the list H . Lemma 10 and Theorem 5 show that neither case occurs.
2. COMPLETE gives each good to the first agent in whose window of $\text{cap}(i)$ consecutive entries of L it occurs. So a good listed twice in H is placed once, and its second entry leaves a slot unused (see Remark 1).
3. The Lean program stores in arrays, filled once, the functions that the specification recomputes: the rankings, the picks, the blocks, the slots, the rotated picks and the output (Section 6).
4. Every loop runs a bounded number of times: Stage R at most n rounds, Phase 1 exactly $|A|$ turns, and the upgrades at most $|A|$ rounds, since each round upgrades a new agent.

4.8 Two examples

We trace K3ALG on two instances. For each we give the state after every stage, summarized in Tables 5 and 7, and check the output by hand. Both were also checked by machine (Appendix C).

Example 2 (owner r). This is the instance of Section 2 (agents $0, \dots, 4$, goods $g_0, \dots, g_8$, values listed there), now traced with the definitions.

Stage R. R1 fails for agent 0 (its favourite g_2 is worth 6, the rest 7), agent 1 (5 against 7), agent 2 (9 against 12) and agent 3 (9 against 15). It applies to agent 4 (its favourite g_5 is worth 6, the rest 0), which leaves with $X_4 = \{g_5\}$. In the next round R1 applies to none of agents 0–3,

Table 4. The state of Stage L. Unless stated otherwise, notions refer to the current pre-allocation (Y, U) .

Object	Definition	Role
$a_i \succ_i b_i \succ_i c_i$	agent i 's three goods by value, ties by index	all decisions of Stage L use only these
pick Y_i	i 's favourite good still unpicked at its turn in Phase 1, or $\perp$	i 's base good
R1 step, insertion step	next agent: one with ≤ 2 goods left, if any; else any agent	R1 priority
block, leader	an insertion step and the R1 steps after it; its inserted agent	needs stay in a block (B2); exposed agents are leaders
J_0	goods never picked in Phase 1	
upgraded, U	agents u with $Y_u = b_u$ that also receive c_u	safe with $\{b_u, c_u\}$ (case P); no slots
junk J	goods that are no pick and no c_u , $u \in U$	must go into slots or to the owner
needs N_i	$\{g \in R_i : g \succ_i Y_i\}$ (R_i if $Y_i = \perp$), for $i \notin U$	goods i lost; must be alone
NA	$\bigcup_{i \notin U} N_i$, the goods needed alone	never in a bundle of ≥ 2 goods
frozen	$i \notin U$ with $Y_i \neq \perp$, $Y_i \in NA$	keeps its pick alone
terminal	$i \notin U$ that is not frozen	may take junk; ends need chains
(UT) valid	no $k \notin U$ with $Y_k = b_k$, $c_k \in J$, $b_k \notin NA$ (V1) $J \cap NA = \emptyset$; (V2) $b_u, c_u \notin NA$, $u \in U$	holds after the upgrades hypothesis of soundness
slots $\text{cap}(i)$	1 (2) for a terminal with (without) a pick; else 0	junk a non-owner may take
S, ω	$\sum_i \text{cap}(i)$; $\omega = J - S$	an owner is needed iff $\omega \geq 1$
base	$\{Y_i\}$ ($\emptyset$) for $i \notin U$; $\{b_i, c_i\}$ for $i \in U$	what i keeps in every completion
owner o	a terminal or upgraded agent that takes the junk left after the other slots	the only bundle of > 2 goods
(OC)	no $x \notin U$, $x \neq o$, $Y_x = a_x$, with $\{b_x, c_x\} \subseteq X_o$	excludes the only threat (case T)
exposed E_o	$x \notin U$, $x \neq o$, $Y_x = a_x$, $\{b_x, c_x\} \subseteq J \cup \text{base}(o)$	the agents (OC) must protect
π_x	$\{b_x, c_x\} \cap J$	one of its goods in a slot protects x
r	last agent of Phase 1 not in U	first candidate owner
HITSET(E)	one junk good per $x \in E$, one shared good for the first pair that meets	exact owner test (Prop. 1)
need chain	distinct $x_0, \dots, x_t$; $x_{s+1} \notin U$ needs Y_{x_s} ; $x_0, \dots, x_{t-1}$ frozen; x_t terminal	gives slots (Thm. 5); rotation (Thm. 6)
B^*, k^*	r 's block; its exposed agent (its leader)	the bad case

Algorithm 1 K3ALG, as formalized in EFX.K3.algoC. Lists are in index order, and “first” means smallest index.

Require: $n \geq 1$ agents, m goods, values $v_i(g) \in \mathbb{N}$

```

1: procedure K3ALG( $v$ )
2:    $A \leftarrow$  all agents;  $M \leftarrow$  all goods;  $X_i \leftarrow \emptyset$  for every agent  $i$ 
3:   while  $|A| \geq 2$ ,  $M \neq \emptyset$ , and rule R1 applies to some agent of  $A$  do  $\triangleright$  Stage R
4:      $i \leftarrow$  the first agent of  $A$  to which R1 applies, with its good  $p$ 
5:      $A \leftarrow A \setminus \{i\}$   $\triangleright i$  leaves
6:     if  $v_i(p) > 0$  then  $X_i \leftarrow \{p\}$ ;  $M \leftarrow M \setminus \{p\}$ 
7:   if  $|A| = 1$  then give  $M$  to the agent of  $A$  and return  $X$ 
8:   if  $M = \emptyset$  then return  $X$ 
9:   return  $X$  together with LBPLUS( $A, M$ )  $\triangleright$  Stage L
10: procedure LBPLUS( $A, M$ )
11:   for  $i \in A$  do  $(a_i, b_i, c_i) \leftarrow$   $i$ 's goods in  $M$  by value, ties by index
12:    $G \leftarrow M$   $\triangleright$  Phase 1
13:   while some agent of  $A$  is unprocessed do
14:      $i \leftarrow$  the first unprocessed agent with  $|\{a_i, b_i, c_i\} \cap G| \leq 2$ 
15:     if there is none,  $i \leftarrow$  the first unprocessed agent
16:      $Y_i \leftarrow$  the first of  $a_i, b_i, c_i$  in  $G$  ( $\perp$  if none);  $G \leftarrow G \setminus \{Y_i\}$ 
17:    $U \leftarrow \emptyset$ ;  $J \leftarrow G$   $\triangleright$  upgrades
18:   while some  $k \in A \setminus U$  has  $Y_k = b_k$ ,  $c_k \in J$  and  $b_k \notin NA(Y, U)$  do
19:      $k \leftarrow$  the first such agent;  $U \leftarrow U \cup \{k\}$ ;  $J \leftarrow J \setminus \{c_k\}$ 
20:   compute  $NA$ , the frozen agents, cap and  $S$ 
21:   if  $|J| \leq S$  then return COMPLETE( $\perp, ()$ )
22:    $r \leftarrow$  the last processed agent not in  $U$ ;  $H \leftarrow \text{HitSet}(E_r)$   $\triangleright$  owner test
23:   if  $|H| \leq S - \text{cap}(r)$  then return COMPLETE( $r, H$ )
24:    $k \leftarrow$  the first agent of  $E_r$  in  $r$ 's block  $\triangleright$  rotation
25:    $C \leftarrow (k)$ 
26:   for each agent  $j$  processed after  $k$ , in processing order do
27:      $\ell \leftarrow$  the last agent of  $C$ 
28:     if  $\ell$  is frozen,  $j \notin U$  and  $j$  needs  $Y_\ell$  then append  $j$  to  $C$ 
29:   each agent of  $C$  after  $k$  takes the pick of its predecessor in  $C$ 
30:    $Y_k \leftarrow b_k$ ;  $U \leftarrow U \cup \{k\}$   $\triangleright k$  also holds  $c_k$ 
31:   recompute  $J$ ,  $NA$ , the frozen agents, cap and  $S$ 
32:   if  $|J| \leq S$  then return COMPLETE( $\perp, ()$ )
33:   return COMPLETE( $k, \text{HitSet}(E_k)$ )
34: procedure COMPLETE( $o, H$ )  $\triangleright o = \perp$ : no owner
35:   give every pick to its picker, and  $c_u$  to every  $u \in U$ 
36:    $L \leftarrow$  the list  $H$  followed by the goods of  $J$  that are not in  $H$ , in index order
37:   for each agent  $i \neq o$ , in index order do
38:     give  $i$  those of the next  $\text{cap}(i)$  entries of  $L$  not given yet
39:   give  $o$  every good of  $J$  not yet given
    
```

Table 5. Example 2, stage by stage.

after	state
Stage R	agent 4 peeled with g_5 ; $A = \{0, 1, 2, 3\}$
Phase 1	order 0, 2, 3, 1; picks $Y_0 = g_2$, $Y_2 = g_1$, $Y_3 = g_8$, $Y_1 = g_6$; blocks $\{0, 2, 3\}$ (leader 0) and $\{1\}$ (leader 1); leftovers $J_0 = \{g_0, g_3, g_4, g_7\}$
upgrades	$U = \{2\}$ (agent 2 receives g_3); $J = \{g_0, g_4, g_7\}$; $NA = \{g_2\}$; frozen: 0; terminals: 1, 3, one slot each; $S = 2$, $\omega = 1$
owner test	$r = 1$; $E_1 = \{0\}$, $\pi_0 = \{g_4\}$; $\text{HITSET}(E_1) = (g_4)$ fits $S - \text{cap}(1) = 1$: owner $r = 1$
completion	$L = (g_4, g_0, g_7)$: g_4 to agent 3, and g_0, g_7 to the owner

so Stage L starts with $A = \{0, 1, 2, 3\}$ and $M = \{g_0, \dots, g_8\} \setminus \{g_5\}$. The rankings are those of the value lists above.

Phase 1 (turns in order):

turn	agent	its goods still unpicked	step	block	pick
1	0	g_2, g_4, g_6 (all)	insertion	1	$g_2 = a_0$
2	2	g_1, g_3	R1	1	$g_1 = b_2$
3	3	g_8	R1	1	$g_8 = b_3$
4	1	g_6, g_7, g_0 (all)	insertion	2	$g_6 = a_1$

At turn 2, agents 2 and 3 have lost g_2 , and agent 2 comes first; at turn 4, agent 1 is the only one left and has all its goods. The leftover goods are $J_0 = \{g_0, g_3, g_4, g_7\}$.

Upgrades. With $U = \emptyset$ the needs are $N_0 = N_1 = \emptyset$ and $N_2 = N_3 = \{g_2\}$, so $NA = \{g_2\}$. Agent 2 has $Y_2 = b_2 = g_1$, $c_2 = g_3 \in J$ and $g_1 \notin NA$: it is upgraded and receives g_3 . Agent 3 has $Y_3 = b_3$, but its $c_3 = g_1$ is agent 2's pick, not junk. The loop stops with $U = \{2\}$ and $J = \{g_0, g_4, g_7\}$.

State. $NA = N_0 \cup N_1 \cup N_3 = \{g_2\}$:

agent	ranking	pick	needs	status	slots
0	$g_2 \succ g_4 \succ g_6$	g_2 (a)	$\emptyset$	frozen (agent 3 needs g_2)	0
1	$g_6 \succ g_7 \succ g_0$	g_6 (a)	$\emptyset$	terminal	1
2	$g_2 \succ g_1 \succ g_3$	g_1 (b), $+g_3$	–	upgraded	0
3	$g_2 \succ g_8 \succ g_1$	g_8 (b)	$\{g_2\}$	terminal	1

So $S = 2 < 3 = |J|$ and $\omega = 1$: an owner is needed.

Owner test. The last processed agent not in U is $r = 1$ (the order is 0, 2, 3, 1), with $S - \text{cap}(1) = 1$ free slot for the others. Agent 0 is exposed for r : it holds $a_0 = g_2$, its $b_0 = g_4$ is junk, and its $c_0 = g_6$ is r 's pick; so $E_1 = \{0\}$ and $\pi_0 = \{g_4\}$. Agent 3 is not exposed, as it holds its b . $\text{HITSET}(E_1) = (g_4)$ has one entry, which fits.

Table 6. Checking Example 2’s output. The largest threat is $\max_{j \neq i} \theta_i(X_j)$ (Section 3.1); the allocation is EFX₀ because it never exceeds $v_i(X_i)$. The last column lists the cases of Lemma 2 that the agent meets.

agent	bundle X_i	$v_i(X_i)$	largest threat	cases
0	$\{g_2\}$	6	5, from $X_3 = \{g_8, g_4\}$	T
1 (owner)	$\{g_6, g_0, g_7\}$	12	0	T, P
2	$\{g_1, g_3\}$	12	0	P, B
3	$\{g_8, g_4\}$	8	7, from $X_2 = \{g_1, g_3\}$	B
4 (peeled)	$\{g_5\}$	6	0	–

Table 7. Example 3, stage by stage.

after	state
Stage R	nobody peeled
Phase 1	order 0, 1, 2, 3; picks $Y_0 = g_2, Y_1 = g_4, Y_2 = g_7, Y_3 = g_0$; blocks $\{0\}$ (leader 0) and $\{1, 2, 3\}$ (leader 1); leftovers $J_0 = \{g_1, g_3, g_5, g_6\}$
upgrades	none: $U = \emptyset, J = J_0; NA = \{g_4, g_7\}$; frozen: 1, 2; terminals: 0, 3, one slot each; $S = 2, \omega = 2$
owner test	$r = 3; E_3 = \{0, 1\}, \pi_0 = \{g_5\}, \pi_1 = \{g_1, g_6\}$, disjoint; $\text{HitSet}(E_3) = (g_5, g_6)$ does not fit $S - \text{cap}(3) = 1$: bad case, $k^* = 1$
rotation	chain $1 \rightarrow 2 \rightarrow 3$: $Y'_2 = g_4, Y'_3 = g_7$, agent 1 holds g_6 and g_1 ; $U' = \{1\}; J' = \{g_0, g_3, g_5\}; NA' = \{g_4\}$; frozen: 2; terminals: 0, 3, one slot each; $S' = 2, \omega' = 1$
completion	owner $k^* = 1; E'_1 = \{0\}, \text{HitSet}(E'_1) = (g_5); L = (g_5, g_0, g_3)$: g_5 to agent 0, g_0 to agent 3, g_3 to the owner

Completion. COMPLETE(1, (g_4)) places $L = (g_4, g_0, g_7)$: agents 0 and 2 have no slots, agent 3 takes g_4 , and the owner takes g_0 and g_7 :

$$X_0 = \{g_2\}, X_1 = \{g_6, g_0, g_7\}, X_2 = \{g_1, g_3\}, X_3 = \{g_8, g_4\}, X_4 = \{g_5\}.$$

The owner’s bundle has $3 = \omega + 2$ goods, as Proposition 2 predicts. Table 6 checks the EFX₀ constraints. The exposed agent 0 is protected by the slot of agent 3, the end of the need chain $0 \rightarrow 3$ in agent 0’s block; this is the slot that the counting in the proof of Theorem 5 provides for agent 0. Placing g_0 in that slot instead would violate EFX₀ (Section 2, Step 5).

Example 3 (a rotation). Four agents share the eight goods g_0 to g_7 . Their positive values are

$$\begin{array}{ll} \text{agent 0: } g_2 \mapsto 8, g_5 \mapsto 7, g_0 \mapsto 3 & \text{agent 2: } g_4 \mapsto 9, g_7 \mapsto 8, g_3 \mapsto 3 \\ \text{agent 1: } g_4 \mapsto 8, g_6 \mapsto 6, g_1 \mapsto 4 & \text{agent 3: } g_4 \mapsto 9, g_7 \mapsto 6, g_0 \mapsto 4 \end{array}$$

Stage R. R1 applies to nobody ($8 < 10$, $8 < 10$, $9 < 11$, $9 < 10$), so Stage L runs on all agents and goods.

Phase 1.

turn	agent	its goods still unpicked	step	block	pick
1	0	g_2, g_5, g_0 (all)	insertion	1	$g_2 = a_0$
2	1	g_4, g_6, g_1 (all)	insertion	2	$g_4 = a_1$
3	2	g_7, g_3	R1	2	$g_7 = b_2$
4	3	g_0	R1	2	$g_0 = c_3$

After turn 1 every waiting agent still has all its goods, so agent 1 is inserted and leads block 2. The leftover goods are $J_0 = \{g_1, g_3, g_5, g_6\}$.

Upgrades. Only agent 2 holds its b , and $c_2 = g_3$ is junk, but agent 3 needs $g_7 = b_2$ alone, so there is no upgrade: $U = \emptyset$. The needs are $N_2 = \{g_4\}$ and $N_3 = \{g_4, g_7\}$, so $NA = \{g_4, g_7\}$:

agent	ranking	pick	needs	status	slots
0	$g_2 \succ g_5 \succ g_0$	g_2 (a)	$\emptyset$	terminal	1
1	$g_4 \succ g_6 \succ g_1$	g_4 (a)	$\emptyset$	frozen (2 and 3 need g_4)	0
2	$g_4 \succ g_7 \succ g_3$	g_7 (b)	$\{g_4\}$	frozen (3 needs g_7)	0
3	$g_4 \succ g_7 \succ g_0$	g_0 (c)	$\{g_4, g_7\}$	terminal	1

So $S = 2 < 4 = |J|$, $\omega = 2$.

Owner test. $r = 3$, and only $S - \text{cap}(3) = 1$ slot is free for the others. Agent 0 is exposed ($b_0 = g_5$ is junk, $c_0 = g_0$ is r 's pick), with $\pi_0 = \{g_5\}$; agent 1 is exposed ($b_1 = g_6$, $c_1 = g_1$ are junk), with $\pi_1 = \{g_1, g_6\}$. The two sets are disjoint, so $\text{HITSET}(E_3) = (g_5, g_6)$ has two entries and does not fit: r is not a valid owner (Proposition 1). Indeed, every completion with owner 3 violates EFX_0 : the single free slot, agent 0's, can keep out of the owner's bundle a good of $\{g_5\}$ or a good of $\{g_1, g_6\}$, but not both, and then agent 0 (worth 8 against 7+3) or agent 1 (worth 8 against 6+4) strongly envies the owner.

This is the bad case of Theorem 5: r 's block is block 2, its leader $k^* = 1$ is exposed and frozen, the sets π_x are disjoint, and every need chain from agent 1 ends at $r = 3$ (there are two: $1 \rightarrow 3$, since agent 3 needs g_4 , and $1 \rightarrow 2 \rightarrow 3$).

Rotation. K3ALG scans agents 2 and 3, which follow agent 1 in the order: agent 2 needs $g_4 = Y_1$ and agent 1 is frozen, so 2 is appended; agent 3 needs $g_7 = Y_2$ and agent 2 is frozen, so 3 is appended. Along the chain $1 \rightarrow 2 \rightarrow 3$ (Figure 2), agent 2 takes g_4 , agent 3 takes g_7 , and agent 1 takes $b_1 = g_6$ with $c_1 = g_1$ and joins U . Agent 3's old pick g_0 becomes junk. The new state has $J' = \{g_0, g_3, g_5\}$, needs $N'_3 = \{g_4\}$ and $N'_0 = N'_2 = \emptyset$, so $NA' = \{g_4\} \subseteq NA$:

Table 8. Checking Example 3’s output, as in Table 6.

agent	bundle X_i	$v_i(X_i)$	largest threat	cases
0	$\{g_2, g_5\}$	15	3, from $X_3 = \{g_7, g_0\}$	T
1 (owner)	$\{g_6, g_1, g_3\}$	10	0	P, B
2	$\{g_4\}$	9	8, from $X_3 = \{g_7, g_0\}$	T
3	$\{g_7, g_0\}$	10	0	P, B

agent	pick after the rotation	needs	status	slots
0	g_2 (a)	$\emptyset$	terminal	1
1	g_6 (b), $+g_1$	–	upgraded	0
2	g_4 (a)	$\emptyset$	frozen (3 needs g_4)	0
3	g_7 (b)	$\{g_4\}$	terminal	1

So $S' = 2 < 3 = |J'|$ and $\omega' = 1$: the owner is $k^* = 1$. Agent 0 is exposed for it ($b_0 = g_5$ and $c_0 = g_0$ are both junk now), with $\pi'_0 = \{g_0, g_5\}$; agent 2 is not, because its $b_2 = g_7$ is agent 3’s pick. $\text{HITSET}(E'_1) = (g_5)$.

Completion. $L = (g_5, g_0, g_3)$: agent 0 takes g_5 , agent 3 takes g_0 , and the owner takes g_3 :

$$X_0 = \{g_2, g_5\}, \quad X_1 = \{g_6, g_1, g_3\}, \quad X_2 = \{g_4\}, \quad X_3 = \{g_7, g_0\}.$$

Table 8 checks it. The exposed agent 0 lies outside r ’s block and is protected by a slot in its own block, its own, as in step (g) of the proof of Theorem 6.

5 Correctness

This section proves that Stage L always returns an EFX₀ allocation with at most one bundle of more than two goods. We prove more than K3ALG needs: the results hold for *every* order of Phase 1 with R1 priority, every order of the upgrades and every need chain used by the rotation. This family of runs is construction LB⁺ (Theorem 7); K3ALG’s smallest-index choices are one member of it, and its tie-breaking plays no role in correctness. Figure 3 shows how the results depend on each other. Readers who consult the repository’s written proof (`proofs/lb_last_step.md` of [9]) will find Theorems 4, 5, 6 and 7 there as Theorems 1’, A, B and C, and Lemma 9 as Lemma 1.

5.1 Invariants of Phase 1

Lemma 7 (Phase 1). *In every run of Phase 1 (any order for (I1) and (I2); an order with R1 priority for the rest):*

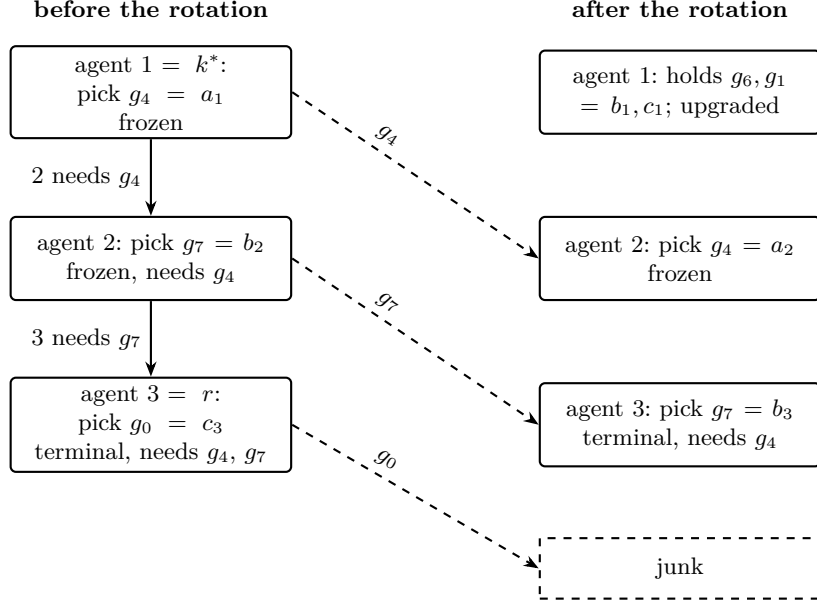


Fig. 2. The rotation of Example 3. Left: the need chain $k^* = 1 \rightarrow 2 \rightarrow 3 = r$ in block 2; each solid arrow points from an agent to an agent that needs its pick. Right: after the rotation, each pick has moved one step down the chain (dashed arrows), k^* holds its b and c , and r 's old pick has become junk. Agent 0, in block 1, is not drawn.

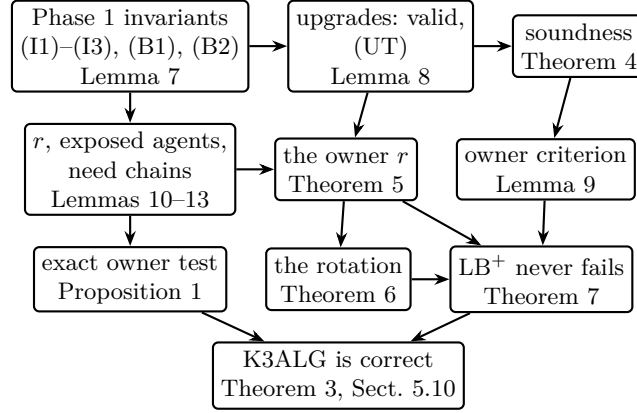


Fig. 3. How the results of Section 5 depend on each other. Theorems 1 and 2 follow from Theorem 7 with peeling (Section 5.8).

- (I1) If $g \in R_i$ and $g \succ_i Y_i$ (or $Y_i = \perp$), then g was picked before i 's turn.
- (I2) If $g \in R_i$ is never picked, then $Y_i \neq \perp$ and $Y_i \succ_i g$.
- (I3) If all of R_i is still unpicked at i 's turn, then i is processed in an insertion step, so it is a leader.
- (B1) When a block starts, every unprocessed agent still has all its goods unpicked.
- (B2) If $g \in R_j$ and $g \succ_j Y_j$ (or $Y_j = \perp$), then g was picked by an agent processed before j in j 's block.

Proof. (I1) At its turn, i took its favourite good among its unpicked goods (or nothing, if none was left), so every good of R_i it ranks higher was already picked; goods leave G only by being picked. (I2) The good g was unpicked at i 's turn, so i had something to pick and took a good it ranks at least as high as g ; that good is not g , which is never picked. (I3) An R1 step processes only an agent with at most two of its goods unpicked. (B1) A block starts with an insertion step, which happens only when no unprocessed agent has at most two goods unpicked. (B2) By (I1), g was picked before j 's turn. Suppose it was picked before j 's block started. At that moment j was unprocessed and $g \in R_j$ was already picked, which contradicts (B1). So g was picked after j 's block started and before j 's turn, by an agent of j 's block. $\square$

In words, (B2) says that everything an agent lost, it lost to earlier agents of its own block. R1 priority enters only through (I3), (B1) and (B2), which Lemmas 11 and 12 below use.

5.2 The state after the upgrades is valid

Lemma 8 (upgrades). *Let Y be the picks of a run of Phase 1, and run the upgrade loop in any order. When it stops, (Y, U) is a valid pre-allocation, and (UT) holds.*

Proof. Pre-allocation. Picks are distinct goods of the pickers. Each upgraded u has $Y_u = b_u$, and c_u was junk when u was upgraded and was then removed from the junk, so the goods c_u are distinct and are nobody's pick.

(V1). The junk is a subset of the leftover goods J_0 . Let $g \in J_0$ and let $i \notin U$ value g . By (I2), $Y_i \succ_i g$, so $g \notin N_i$. Hence no leftover good is needed alone, for any U .

(V2). For $u \in U$, the good c_u is a leftover good, so $c_u \notin NA$ by the argument for (V1). When u was upgraded, $b_u \notin NA$ for the set U of that

moment. Later upgrades only remove agents' needs from the union NA , so $b_u \notin NA$ still holds at the end.

(*UT*) is the stopping condition of the loop. $\square$

In Lean: `EFX.LB.upFinal_valid`. In a valid pre-allocation, every good $g \in NA$ is the pick of an agent outside U : it is not junk by (V1), and it is not b_u or c_u by (V2). That agent's pick is in NA , so it is frozen.

5.3 Soundness of completions

Theorem 4 (soundness). *Let (Y, U) be a valid pre-allocation, and let X be a completion of it, without owner or with an owner o , that satisfies the owner constraint (OC). Then X is EFX_0 for every balanced valuation consistent with the rankings, and every bundle except the owner's has at most two goods.*

Proof. Sizes. A frozen agent other than the owner holds its pick, one good. An upgraded agent other than the owner holds $\{b_i, c_i\}$. A terminal other than the owner holds its base and at most $\text{cap}(i)$ junk goods, and $|\text{base}(i)| + \text{cap}(i) = 2$. So only X_o can have more than two goods.

Bundles of two or more goods contain no good needed alone. Let $|X_j| \geq 2$. Then j is not frozen (unless it is the owner, which is never frozen), so X_j consists of: j 's pick if j is a terminal, which is not in NA by the definition of a terminal; or b_j and c_j if $j \in U$, which are not in NA by (V2); and junk goods, which are not in NA by (V1). So $X_j \cap NA = \emptyset$.

Each agent is safe. Fix an agent i and another bundle X_j . If $|X_j| \leq 1$, its threat is 0 (Lemma 1(a)). So let $|X_j| \geq 2$; then $X_j \cap NA = \emptyset$. Let $h \in X_j$.

- $i \in U$. Then $X_i \supseteq \{b_i, c_i\}$, so X_j meets R_i at most in a_i , and $v_i(X_j \setminus \{h\}) \leq v_i(a_i) \leq v_i(b_i) + v_i(c_i) \leq v_i(X_i)$, by balance.
- $i \notin U$ and $Y_i = \perp$. Then $R_i = N_i \subseteq NA$, so X_j contains no good of R_i , and $v_i(X_j \setminus \{h\}) = 0 \leq v_i(X_i)$.
- $i \notin U$ with a pick. A good of $R_i \cap X_j$ is not in $N_i \subseteq NA$, and it is not Y_i , which is in X_i . So i ranks it below Y_i , and by consistency it is worth at most $v_i(Y_i) \leq v_i(X_i)$. If $R_i \cap X_j$ has at most one good, then $v_i(X_j \setminus \{h\}) \leq v_i(X_j) \leq v_i(Y_i) \leq v_i(X_i)$. Otherwise $R_i \cap X_j$ consists of two goods ranked below Y_i , which forces $Y_i = a_i$ and $R_i \cap X_j = \{b_i, c_i\}$. If $|X_j| = 2$, then $X_j = \{b_i, c_i\}$, and $X_j \setminus \{h\}$ is one of these goods, worth at most $v_i(b_i) \leq v_i(a_i) \leq v_i(X_i)$. If $|X_j| \geq 3$, then $j = o$, and $i \notin U$, $i \neq o$, $Y_i = a_i$, $\{b_i, c_i\} \subseteq X_o$: exactly what (OC) excludes. $\square$

The proof uses only agent i 's own ranking and balance, so the output depends on the values only through the rankings. It does not cite Lemma 2. In Lean: `EFX.LB.Valid.sound`.

5.4 The owner criterion

Lemma 9 (owner criterion). *Let (Y, U) be valid. (a) If $\omega \leq 0$, a completion without owner exists, and it satisfies (OC). (b) If $\omega \geq 1$ and o is a valid owner with the set H , then every completion with owner o that puts every good of H into some slot (that is, $H \subseteq \bigcup_i C_i$) satisfies (OC), and such a completion exists.*

Proof. (a) If $|J| \leq S$, the junk can be split into sets C_i of at most $\text{cap}(i)$ goods; (OC) is vacuous without owner. (b) *Existence.* The terminals other than o have $S - \text{cap}(o) \geq |H|$ slots in all, so the goods of H fit into them; the rest of the junk may fill further slots or go to o . (OC). Suppose some $x \notin U$, $x \neq o$, with $Y_x = a_x$, has $\{b_x, c_x\} \subseteq X_o = \text{base}(o) \cup (J \setminus C)$, where $C = \bigcup_i C_i$. Then $\{b_x, c_x\} \subseteq J \cup \text{base}(o)$, so $x \in E_o$, and H contains a good h of $\{b_x, c_x\}$. But $h \in H \subseteq C$, so h is junk placed in a slot: it is neither in $J \setminus C$ nor in $\text{base}(o)$, whose goods are not junk. So $h \notin X_o$, a contradiction. $\square$

K3ALG's $\text{COMPLETE}(o, H)$ is such a completion whenever H consists of junk goods, meets every pair $\{b_x, c_x\}$ with $x \in E_o$, and has at most $S - \text{cap}(o)$ entries: the first $|H|$ entries of the list L lie within the windows of the agents other than o , whose total length is $S - \text{cap}(o)$, so every good of H is placed in a slot. Without owner and with $|J| \leq S$, L lists the junk without repetition and every good of it is placed. In Lean: `EFX.LB.complete_some` and `EFX.LB.complete_none`.

5.5 The candidate owner and exposed agents

From now on, until Proposition 1, fix a run of Phase 1 with R1 priority and of the upgrades, let (Y, U) be the resulting valid pre-allocation (Lemma 8), and assume $\omega \geq 1$.

Lemma 10 (the candidate r). *(A0) Some agent is not upgraded; let r be the last processed agent that is not upgraded. (A1) r is a terminal. (A2) Every agent processed after r is upgraded.*

Proof. (A0) The first processed agent is a leader, so it picks its top a , while every upgraded agent picked its b . (A1) If $Y_r = \perp$, then r is a

terminal by definition. Otherwise suppose $Y_r \in NA$: some $j \notin U$ needs Y_r . Then $j \neq r$ (an agent does not need its own pick), and by (I1) Y_r was picked before j 's turn; it was picked at r 's turn, so j was processed after r . As $j \notin U$, this contradicts the choice of r . So r is not frozen. (A2) This is the choice of r . $\square$

Let B^* denote r 's block. It is the last block, since the leader of any later block would pick its top and so would not be upgraded, yet would come after r ; this is not needed below. In Lean: `EFX.LB.lastOut_exists`, `EFX.LB.lastOut_terminal`; and for the last block, `EFX.LB.lastOut_lastBlock` (no agent's block exceeds r 's, and r 's block is that of the last processed agent), from the general form `EFX.LB.blk_le_lastOut`, which needs only that no upgraded agent picks its top.

Lemma 11 (exposed agents are leaders). *Let $x \in E_r$. Then x was processed before r , x is a leader, and $\pi_x \neq \emptyset$. Consequently each block contains at most one agent of E_r ; in particular $|E_r \cap B^*| \leq 1$.*

Proof. As $x \notin U$ and $x \neq r$, x was processed before r by (A2). At x 's turn, $a_x = Y_x$ was unpicked, since x picked it; every junk good was unpicked, since junk is never picked; and Y_r was unpicked, since r picked it later. As $\{b_x, c_x\} \subseteq J \cup \text{base}(r) = J \cup \{Y_r\}$, all of R_x was unpicked at x 's turn, and x is a leader by (I3). A block has only one leader. Finally, $b_x \neq c_x$, so at most one of them is Y_r , and the other is junk: $\pi_x \neq \emptyset$. $\square$

In Lean: `EFX.LB.exposed_lead`.

Lemma 12 (need chains). *Let x be frozen. Then there is a need chain from x , and every need chain from x consists of agents of x 's block, each processed after the previous one. So every frozen agent x has a terminal $\tau(x)$ in its block that ends a need chain from x ; for a terminal x let $\tau(x) = x$. In both cases $\text{cap}(\tau(x)) \geq 1$.*

Proof. If x is frozen, some $j \notin U$ needs Y_x , and $j \neq x$. By (B2), Y_x was picked by an agent processed before j in j 's block; that agent is x , the picker of Y_x . So j lies in x 's block and is processed after x . If j is frozen, repeat from j . Each step moves strictly later in the processing order, so the agents are distinct, and the process stops, at a terminal, within x 's block. The same argument shows that any need chain from x stays in x 's block. A terminal has $\text{cap} \geq 1$. $\square$

In Lean: `EFX.LB.chainEnd_spec`. The next lemma is the heart of the argument; Figure 4 illustrates it.

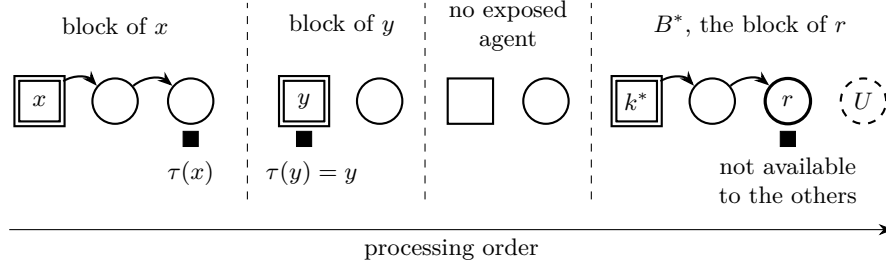


Fig. 4. The counting of Lemma 13 and Theorem 5. Squares are leaders, circles other agents; double borders mark agents exposed for r , and black squares mark slots. Exposed agents are leaders (Lemma 11), so they lie in different blocks. Each exposed agent outside B^* reaches a terminal with a slot by a need chain inside its own block (arrows; Lemma 12). Only the exposed leader k^* of B^* may lack such a slot: when all its need chains end at r and no two exposed agents share a junk good, we are in the bad case. Agents processed after r are upgraded.

Lemma 13 (counting). *The terminals $\tau(x)$, for $x \in E_r \setminus B^*$, are pairwise distinct, lie outside B^* , and differ from r . Hence*

$$S - \text{cap}(r) \geq |E_r \setminus B^*| \geq |E_r| - 1.$$

Proof. By Lemma 11, different agents of E_r lie in different blocks, and by Lemma 12, $\tau(x)$ lies in x 's block. So the terminals $\tau(x)$, $x \in E_r \setminus B^*$, lie in pairwise different blocks other than B^* ; in particular they are distinct and none is $r \in B^*$. Each has at least one slot, so the terminals other than r have at least $|E_r \setminus B^*|$ slots. Finally $|E_r \cap B^*| \leq 1$ by Lemma 11. $\square$

5.6 The owner r

Theorem 5 (the owner r). *Let (Y, U) be the state after a run of Phase 1 with R1 priority and of the upgrades, with $\omega \geq 1$. Then r is a valid owner, unless the following bad case holds:*

- (i) r 's block B^* contains an agent k^* exposed for r , and k^* is frozen;
- (ii) every need chain from k^* ends at r ; and
- (iii) the sets π_x , $x \in E_r$, are pairwise disjoint.

Proof. Let H consist of one good of each set π_x ($x \in E_r$; these sets are nonempty by Lemma 11), choosing a common good when two of the sets meet; so $|H| \leq |E_r|$, and $|H| \leq |E_r| - 1$ if two sets meet. H is a set of junk goods meeting every π_x , so r is a valid owner as soon as $|H| \leq S - \text{cap}(r)$. We use Lemma 13.

- If $E_r \cap B^* = \emptyset$, then $|H| \leq |E_r| = |E_r \setminus B^*| \leq S - \text{cap}(r)$.
- Otherwise $E_r \cap B^* = \{k^*\}$ for one agent k^* (Lemma 11), $k^* \neq r$ because $r \notin E_r$, and $|E_r \setminus B^*| = |E_r| - 1$.
 - If k^* is a terminal, or some need chain from k^* ends at a terminal other than r , then that terminal, $\tau(k^*)$, lies in B^* and differs from r and from all the terminals $\tau(x)$, $x \in E_r \setminus B^*$, which lie outside B^* . It has a slot, so $S - \text{cap}(r) \geq |E_r| \geq |H|$.
 - If two of the sets π_x meet, then $|H| \leq |E_r| - 1 = |E_r \setminus B^*| \leq S - \text{cap}(r)$.

If none of these cases applies, then k^* exists, is not a terminal, and is not upgraded (it is exposed), so it is frozen; every need chain from k^* ends at r ; and the sets π_x are pairwise disjoint. This is the bad case. $\square$

In Lean: `EFX.LB.theoremA_invalid` (if r is not a valid owner, the bad case holds). Example 2 is the first case of the proof, and Example 3 is the bad case.

5.7 The rotation

Assume the bad case, and write $k = k^*$. Fix any need chain $x_0, x_1, \dots, x_t$ from $x_0 = k$, with $t \geq 1$; one exists by Lemma 12, and it ends at $x_t = r$ by (ii). Define the rotated pre-allocation $P' = (Y', U')$:

- $Y'_{x_s} = Y_{x_{s-1}}$ for $1 \leq s \leq t$: every later agent of the chain takes the good it needed from its predecessor;
- $Y'_k = b_k$ and $U' = U \cup \{k\}$: agent k gives up a_k and takes b_k and c_k ;
- every other agent keeps its pick and its upgrade status.

Let $W = J \cup \{Y_r\}$ (just J if $Y_r = \perp$). Since $k \in E_r$, $\{b_k, c_k\} \subseteq W$. The new junk is $J' = W \setminus \{b_k, c_k\}$: the pick of r , if any, is released, and every pick of P other than Y_r is still a pick in P' , possibly of another agent of the chain. Primes denote the notions of P' : N'_i , NA' , cap' , S' , ω' and E'_k (exposure for the owner k in P'). Note that $J' \cup \{b_k, c_k\} = W$.

Theorem 6 (the rotation). *In the bad case, P' is a valid pre-allocation with $NA' \subseteq NA$. If $\omega' \leq 0$, it has a completion without owner. If $\omega' \geq 1$, then every $x \in E'_k$ has $\pi'_x = \{b_x, c_x\} \cap J' \neq \emptyset$ and $|E'_k| \leq S'$; in particular k is a valid owner of P' , and $\text{HITSET}(E'_k)$ is a witness.*

Proof. (a) P' is a pre-allocation. Each x_s ($s \geq 1$) needed $Y_{x_{s-1}}$, so $Y'_{x_s} \in R_{x_s}$. The picks of P' are the picks of P other than Y_r , rearranged along the chain, together with b_k ; since $b_k \in W$, it is not a pick of P other than

Y_r , so no good is picked twice. Also $c_k \in W$ and $c_k \neq b_k$, so c_k is not a pick of P' ; and it differs from every c_u , $u \in U$, which is not in W . Finally $Y'_k = b_k$, as the definition requires for $k \in U'$.

(b) $NA' \subseteq NA$. For $s \geq 1$, x_s ranks Y'_{x_s} above Y_{x_s} (or $Y_{x_s} = \perp$), so $N'_{x_s} \subseteq N_{x_s}$. Agent k had $N_k = \emptyset$, since $Y_k = a_k$, and is now in U' . Every other agent keeps its needs.

(c) *Validity*. (V1): $J' \subseteq W$, $J \cap NA = \emptyset$ by (V1) for P , and $Y_r \notin NA$ by (A1); so $J' \cap NA' = \emptyset$. (V2): for $u \in U$, $b_u, c_u \notin NA \supseteq NA'$; for k , $b_k, c_k \in W$, and $W \cap NA = \emptyset$ as just shown.

(d) *Terminals outside B^* survive*. The chain lies in B^* (Lemma 12). Let x be a terminal of P outside B^* . It keeps its pick and stays outside U' , and $Y'_x = Y_x \notin NA \supseteq NA'$, so it is a terminal of P' with the same number of slots.

(e) $E'_k \subseteq E_r \setminus \{k\}$. By definition E'_k is the set of $x \notin U'$, $x \neq k$, with $Y'_x = a_x$ and $\{b_x, c_x\} \subseteq J' \cup \{b_k, c_k\} = W$. Let $x \in E'_k$.

- If x is not on the chain, then $Y_x = Y'_x = a_x$, $x \notin U$, $x \neq r$, and $\{b_x, c_x\} \subseteq W = J \cup \text{base}(r)$. So $x \in E_r$.
- $x = x_s$ with $1 \leq s \leq t-1$ is impossible. Since $a_x = Y'_x \succ_x Y_x$, the old pick Y_x is b_x or c_x ; but Y_x is a pick of P other than Y_r , so $Y_x \notin W$, contradicting $\{b_x, c_x\} \subseteq W$.
- $x = r$ is impossible. Since r needs $Y_{x_{t-1}}$, $Y_r \neq a_r$, so there are three cases. If $Y_r = \perp$, then $b_r \in N_r$ was picked by another agent (I1), so $b_r \notin W$. If $Y_r = c_r$, the same holds for b_r . If $Y_r = b_r$, then (UT) applies to r : $r \notin U$, and $b_r \notin NA$ by (A1), so $c_r \notin J$; and $c_r \neq Y_r$, so $c_r \notin W$. In each case $\{b_r, c_r\} \not\subseteq W$.

(f) *Every exposed pair has a junk good*. Let $x \in E'_k$, and suppose $\{b_x, c_x\} \subseteq \{b_k, c_k\}$, so the two pairs are equal. By (e), $x \in E_r \setminus \{k\}$, and then $\pi_x = \{b_x, c_x\} \cap J = \{b_k, c_k\} \cap J = \pi_k$, which is nonempty (Lemma 11); this contradicts the disjointness (iii) of the bad case. So $\{b_x, c_x\}$ has a good outside $\{b_k, c_k\}$, which lies in J' because $\{b_x, c_x\} \subseteq J' \cup \{b_k, c_k\}$: $\pi'_x \neq \emptyset$.

(g) *Counting*. By (e), $|E'_k| \leq |E_r| - 1 = |E_r \setminus B^*|$, since $E_r \cap B^* = \{k\}$. The terminals $\tau(x)$, $x \in E_r \setminus B^*$, are distinct and lie outside B^* (Lemma 13), so by (d) they are terminals of P' with at least one slot each. Hence $|E'_k| \leq S' = S' - \text{cap}'(k)$, as $k \in U'$ has no slots.

Finally, if $\omega' \leq 0$, Lemma 9(a) applies. If $\omega' \geq 1$, then one junk good from each π'_x , $x \in E'_k$, forms a set of at most $|E'_k| \leq S' - \text{cap}'(k)$ junk goods meeting every exposed pair, so k is a valid owner. The list $\text{HITSET}(E'_k)$ has at most $|E'_k|$ entries, each a junk good (the shared good of the first meeting pair, or one(z) $\in \pi'_z$, which is junk because $\pi'_z \neq \emptyset$), and it meets every pair. $\square$

In Lean: `EFX.LB.theoremB`, for any need chain. Figure 2 shows a rotation, and Example 3 illustrates (e)–(g): after the rotation, only agent $0 \in E_r \setminus B^*$ is exposed for k^* , and the terminals outside B^* keep their slots.

5.8 Construction LB^+ never fails

Construction LB^+ is the following family of runs on an instance in which every agent values exactly three goods, ranked, and is balanced.

1. Run Phase 1 in any order with R1 priority, then the upgrades in any order.
2. If $\omega \leq 0$, return any completion without owner.
3. Otherwise, if r is a valid owner, return any completion with owner r that satisfies (OC).
4. Otherwise rotate along any need chain from k^* to r . If $\omega' \leq 0$, return any completion of P' without owner; otherwise return any completion of P' with owner k^* that satisfies (OC).

Theorem 7 (LB^+ never fails). *For every instance in which every agent values exactly three goods and is balanced, and for every run of LB^+ , an output exists (every step above is possible), and every output is a complete allocation that is EFX_0 for every balanced valuation consistent with the rankings, in which at most one bundle has more than two goods.*

Proof. Step 1 gives a valid pre-allocation (Lemma 8). In step 2, a completion without owner exists (Lemma 9(a)). In step 3, a completion with owner r satisfying (OC) exists (Lemma 9(b)). In step 4, r is not a valid owner, so the bad case holds (Theorem 5): k^* exists, a need chain from k^* exists, and every such chain ends at r . By Theorem 6, P' is valid, and either $\omega' \leq 0$ and a completion without owner exists, or k^* is a valid owner of P' and a completion with owner k^* satisfying (OC) exists (Lemma 9(b)). Every returned completion is of a valid pre-allocation and satisfies (OC), vacuously when there is no owner, so Theorem 4 applies. $\square$

In Lean: `EFX.LB.lbPlusRun_sound` (every output is sound, over all runs), `EFX.LB.lbPlusOut_exists` (an output exists) and `EFX.LB.lbPlusOut_exists_chain` (in the rotation branch, every need chain from k^* to r gives one).

Proof (of Theorem 2). Rank each agent's three goods by value, ties by index; the valuation is consistent with these rankings and balanced. Run

LB⁺ (for instance K3ALG's Stage L, which is one of its runs). By Theorem 7, the output is EFX₀ for this valuation, and at most one bundle has more than two goods. The argument never uses that values are integers. $\square$

Proof (of Theorem 1). By induction on the number of agents $n \geq 1$. With one agent, give it everything. With no goods, give everyone nothing. Otherwise, if rule R1 applies to some agent i , with bundle P ($P = \{p\}$ or $\emptyset$), the instance without i and P is again 3-relevant and has $n - 1 \geq 1$ agents, so it has an EFX₀ allocation by induction, and Lemma 3 extends it. If R1 applies to no agent, then by Lemma 4 every agent values exactly three goods and is strictly balanced, and Theorem 2 applies. $\square$

The Lean proofs of Theorems 1 and 2 are over natural-number values, `EFX.target` and `EFX.LB.corollaryD`; Lemma 6 transfers them to any linearly ordered cancellative commutative monoid (`EFX.target_ordered`, `EFX.corollaryD_ordered`; Section 7). The Lean proof of Theorem 1 goes through the reduction to cores of Section 8 (`EFX.core_reduction`), which also removes goods nobody values and peels by rule R2; the proof above shows that rule R1 alone suffices, which is the route K3ALG takes and whose Lean form is `EFX.K3.reduce_sound`.

5.9 The owner test is exact

To apply Theorem 7, an algorithm must decide whether r is a valid owner. In general, deciding whether some set of at most s junk goods meets every set π_x is a minimum hitting set problem; since each π_x has at most two goods, it is vertex cover on the graph whose vertices are the junk goods and whose edges are the sets π_x , which is NP-hard in general. For the particular owner r the counting of Lemma 13 makes it easy.

Proposition 1 (the owner test). *Let (Y, U) be the state after a run of Phase 1 with R1 priority and of the upgrades, with $\omega \geq 1$. Then r is a valid owner if and only if*

$$|E_r| \leq S - \text{cap}(r) \quad \text{or} \quad \text{two of the sets } \pi_x \ (x \in E_r) \text{ meet.}$$

In either case $\text{HITSET}(E_r)$ is a witness: it lists junk goods, meets π_x for every $x \in E_r$, and has at most $S - \text{cap}(r)$ entries. Consequently, r is a valid owner if and only if $\text{HITSET}(E_r)$ has at most $S - \text{cap}(r)$ entries.

Proof. Let τ be the smallest number of junk goods that meet every set π_x , $x \in E_r$. By definition, r is a valid owner if and only if $\tau \leq S - \text{cap}(r)$.

- $\tau \leq |E_r|$, and $\tau \leq |E_r| - 1$ if and only if two of the sets meet. All the sets are nonempty (Lemma 11), so one good from each gives $\tau \leq |E_r|$. If $g \in \pi_x \cap \pi_y$ for $x \neq y$, then g together with one good from each other set meets all of them, with at most $|E_r| - 1$ goods. If the sets are pairwise disjoint, a set meeting all of them contains a different good of each, so $\tau = |E_r|$.
- *Sufficiency.* If $|E_r| \leq S - \text{cap}(r)$, then $\tau \leq |E_r| \leq S - \text{cap}(r)$. If two sets meet, then $\tau \leq |E_r| - 1 \leq S - \text{cap}(r)$ by Lemma 13.
- *Necessity.* If no two sets meet, then $\tau = |E_r|$, so $\tau \leq S - \text{cap}(r)$ gives $|E_r| \leq S - \text{cap}(r)$.
- *The witness.* Every entry of $\text{HitSet}(E_r)$ is junk: the shared good g is in $\pi_x \cap \pi_y$, and $\text{one}(z)$ is b_z if b_z is junk and otherwise c_z , which is then junk because $\pi_z \neq \emptyset$. The list meets every π_x (x and y through g , every other z through $\text{one}(z)$), and it has $|E_r| - 1$ entries if two sets meet and $|E_r|$ otherwise. By the previous items, in both cases of the condition it has at most $S - \text{cap}(r)$ entries, and if the condition fails it has $|E_r| > S - \text{cap}(r)$ entries. $\square$

In Lean: `EFX.LB.validOwner_iff`, with the counting `EFX.LB.hitSet_fits` and the length `EFX.LB.hitSet_length`. The simpler test “ $|E_r| \leq S - \text{cap}(r)$ ”, one good per exposed agent without sharing, is not enough: it rejects some valid owners outside the bad case, where the rotation’s guarantees do not hold (a smallest configuration is recorded in the repository [9]). After the rotation no test is needed, by Theorem 6.

5.10 Correctness of K3ALG

Proof (of Theorem 3, correctness). Let every agent value at most three goods. Stage R peels agents by rule R1, so by Lemma 3 and induction it suffices that Stage L returns an EFX_0 allocation of the agents A and goods M it receives (when Stage R returns directly, what remains is one agent, which takes everything, or no goods, and its allocation is trivially EFX_0). When Stage L starts, every agent of A values exactly three goods of M and is strictly balanced (Lemma 4), and the computed rankings are consistent with the values. We check that Stage L is a run of LB^+ .

- Phase 1 serves the first agent with at most two goods left, if any, which is an order with R1 priority; the upgrades run in index order.
- If $|J| \leq S$, it returns a completion without owner (step 2).
- Otherwise its test $|\text{HitSet}(E_r)| \leq S - \text{cap}(r)$ holds exactly when r is a valid owner (Proposition 1). If it holds, $\text{COMPLETE}(r, \text{HitSet}(E_r))$

is a completion satisfying (OC) (Proposition 1 and the remark after Lemma 9): step 3.

- If the test fails, r is not a valid owner, so the bad case holds (Theorem 5). Then $E_r \cap B^* = \{k^*\}$, so the first agent of E_r in r 's block is k^* . The scan builds a need chain from k^* : each appended agent is outside U and needs the pick of the frozen agent before it, all are distinct (they are scanned in order), and while the last agent is frozen, the agent that needs its pick comes later in the order (Lemma 12), so the scan finds it; hence the chain ends at a terminal, which is r by (ii). The rotation is then that of Theorem 6. If $|J'| \leq S'$, K3ALG completes without owner; otherwise $\text{HITSET}(E'_{k^*})$ is a witness that k^* is a valid owner (Theorem 6), and $\text{COMPLETE}(k^*, \text{HITSET}(E'_{k^*}))$ satisfies (OC): step 4.

By Theorem 7, Stage L's output is EFX₀. □

In Lean: `EFX.K3.algo_efx0`, via `EFX.K3.algo_eq_spec` (the counted program computes the specification) and `EFX.K3.algoSpec_efx0`. The two “impossible” branches of the listing (no r , no k^*) are indeed never taken, by Lemma 10 and Theorem 5.

5.11 The size of the large bundle

Proposition 2 (size of the large bundle). *Let (Y, U) be a valid pre-allocation of an instance with n agents and m goods in which every agent values exactly three goods. Then $\omega = m - 2n + |NA|$. Every completion with owner o gives o at least $\omega + 2$ goods, and exactly $\omega + 2$ when all slots of the terminals other than o are full.*

Proof. In a valid pre-allocation, each good of NA is the pick of an agent outside U , which is then frozen (Section 5.2), and each frozen agent's pick is in NA ; so the frozen agents F satisfy $|F| = |NA|$. Let T_1 and T_0 be the terminals with and without a pick. Every good is a pick, a good c_u , or junk, so $m = (|U| + |F| + |T_1|) + |U| + |J|$, while $S = |T_1| + 2|T_0|$ and $n = |U| + |F| + |T_1| + |T_0|$. Hence

$$\omega = |J| - S = m - 2|U| - |F| - 2|T_1| - 2|T_0| = m - 2n + |F| = m - 2n + |NA|.$$

The other terminals receive at most $S - \text{cap}(o)$ junk goods, exactly that many when their slots are full, so the owner receives its base and at least $|J| - (S - \text{cap}(o)) = \omega + \text{cap}(o)$ junk goods, with equality when the other slots are full; and $|\text{base}(o)| + \text{cap}(o) = 2$ for every terminal and every upgraded agent. □

An owner is needed only when $\omega \geq 1$, but the count and both bounds hold for every ω . The rotation does not increase NA (Theorem 6), so it does not enlarge the large bundle. In Example 2, $m - 2n + |NA| = 8 - 8 + 1 = 1 = \omega$ for the Stage-L instance, and the owner holds 3 goods. In Lean: `EFX.LB.largeBundle_size`, from `EFX.LB.omega_eq`, `EFX.LB.numFrozen_eq_numNA`, `EFX.LB.Completion.owner_length_ge` and `EFX.LB.Completion.owner_length_eq`; for the rotation, `EFX.LB.BadCase.omega_le`.

Remark 1 (a repeated good). K3ALG's completion fills every slot of the terminals other than the owner when HITSET lists no good twice, and then Proposition 2 gives the size of its large bundle, $\omega + 2$ (in Lean, `EFX.LB.complete_owner_length`, for $\omega \geq 1$ and the hypotheses of Lemma 9). A good can be listed twice, and then a slot stays empty and the owner receives more goods; the output is still EFX_0 by Lemma 9. For example, let four agents value g_0 at 3, and agent i also value g_{2i+1} at 4 and g_{2i+2} at 2 ($i = 0, \dots, 3$, nine goods). All four agents are leaders and pick their tops, $NA = \emptyset$, every agent is a terminal with one slot, $S = 4$ and $|J| = 5$, so $\omega = 1$. The owner is $r = 3$, with $E_3 = \{0, 1, 2\}$ and $\text{HITSET}(E_3) = (g_0, g_0)$: the pair of agents 0 and 1 shares g_0 , and agent 2's junk b is g_0 again. So $L = (g_0, g_0, g_2, g_4, g_6, g_8)$: agent 0 receives g_0 , agent 1's window holds only the repeated g_0 , agent 2 receives g_2 , and the owner receives g_4, g_6 and g_8 besides its pick g_7 : four goods in all, which is $\omega + 3$. Lean checks this run of K3ALG, its state and the bundle sizes by `decide`, and that the output is EFX_0 (`EFX.K3.Examples.repeatedGood_state`, `EFX.K3.Examples.repeatedGood_algo`).

6 Running Time

Running-time claims come in three layers, which we keep apart.

- (i) *Lean-checked*: on every input, the formalized program K3ALG performs, in the unit-cost model of Section 6.2, at most $400(n + m + 1)^4$ operations (Theorem 8). It takes the valuation as a function v , read as a dense $n \times m$ table at one unit per value.
- (ii) *Lean-checked*: the same program performs at most $n^4 + 20n^3 + 25n^2m + 124n^2 + 47nm + 119n + 22m + 3 \leq 270(n^4 + n^2m)$ operations, in the same cost model (Theorem 9, Section 6.5).
- (iii) *A separate implementation*: an optimized Python program, not verified, for which a written analysis, not formalized, gives $O((n+m) \log n)$

operations after the input is read, and whose speed is measured (Section 6.5). Reading a dense $n \times m$ table already costs nm operations, so (iii) excludes the reading of the input; the implementation receives each agent's relevant goods as a list.

Section 6.1 explains why the owner test is cheap; Section 6.2 describes layer (i) and its cost model; Section 6.3 gives a written, not formalized, corollary for the bit model, and Section 6.4 Lean-checked corollaries for rational and real values; and Section 6.5 gives layers (ii) and (iii).

6.1 The owner test in polynomial time

The only step of LB^+ that looks expensive is the owner test, since for an arbitrary candidate owner it is a vertex-cover question (Section 5.9). Proposition 1 removes the issue for the candidate r : r is a valid owner if and only if the explicit list $\text{HITSET}(E_r)$ has at most $S - \text{cap}(r)$ entries. Computing $\text{HITSET}(E)$ means finding the first pair of agents of E whose sets π share a junk good: $O(|E|^2)$ pair checks, so $O(|E|^2 + m)$ steps with a junk flag per good, or $O(|E|^2 m)$ as formalized, where membership in J is a list scan. With an index from each junk good to the exposed agents whose pair contains it, $O(|E| + |J|)$ steps suffice. After the rotation no test is needed (Theorem 6). Whether the test is hard for an arbitrary owner, on the graphs that arise here, is open; LB^+ never asks.

6.2 Layer (i): the Lean-checked operation bound

The program and its cost. In Lean, K3ALG is written in a *cost monad*, EFX.Timed : a value of type $\text{Timed } \alpha$ is a value of type α paired with a natural number, the count; `tick  $k$` adds k to the count, and sequencing adds the counts of the steps actually evaluated. The algorithm is EFX.K3.algoC , of type Timed Alloc , and

```
def EFX.K3.algo (I : Inst) (hn : 0 < I.n) : I.Alloc :=
  (algoC I hn).val
```

is its value. So the machine-checked bound below is a bound for the very program whose value is the allocation, not for a separately written cost function. Each counted definition has a *value lemma* saying that it computes the corresponding uncounted definition of Section 5's formalization; for the whole program this is $\text{EFX.K3.algo_eq_spec}$.

Theorem 8 (running time; `EFX.K3.algoC_cost`). *For every instance with $n \geq 1$ agents and m goods, with any natural-number values (no assumption on the number of relevant goods), $(\text{algoC } \text{I } \text{hn}).\text{cost} \leq 400(n + m + 1)^4$; also $\leq 6400(n + m)^4$ (`EFX.K3.algoC_cost'`).*

The cost model. One unit is counted for each of the following:

- one step of a list traversal (a cell visited by `map`, `filter`, `any`, `find`, a sum, `length`, `append`, `take`, `drop`, membership or `erase`, or by the recursions of Phase 1, the blocks, the need chain, the rotation, the completion and the search for r), together with the comparison made there;
- one comparison of two agents or two goods;
- one read of an input value $v_i(g)$ (the input is an $n \times m$ matrix);
- one comparison or addition of two natural numbers, of any size;
- one read of a *table*: the rankings a_i, b_i, c_i , the picks, the blocks, the slots and the output are arrays filled once (`EFX.Timed.mkTable`); filling a table costs the evaluation of its entries plus one unit per entry.

Straight-line code with a bounded number of these operations is charged by a single `tick` k , with k at least that number; for example, sorting one agent's three goods costs `tick 10` (at most four list cells, three value reads and three comparisons), and computing the rank of a good for an agent costs `tick 6` (three table reads and three comparisons). Not counted: building pairs, options and list cells that a counted step produces, and matching on them; Boolean connectives on computed Booleans (a constant number per counted step); building closures; and the size of numbers, which Corollary 3 accounts for.

The accounting discipline. Four rules make the count meaningful.

1. Every list traversal goes through a charged combinator, such as `mapC`, `filterC`, `anyC`, `findC`, `memC`, `eraseC`, `sumMapC` or `lengthC`: one unit per cell visited, plus the cost of the element function. The recursions of Phase 1, the blocks, the chain, the rotation and the completion charge one unit per step.
2. Every call of an uncounted function sits under a `tick` that bounds its work: `sort3C` is `tick 10`, `rankC` is `tick 6`, and `finRangeC` k is `tick k`.
3. No recursion over lists runs outside the monad, and each counted definition has a value lemma, so every loop of the specification is a counted loop.

4. Every table is filled once by `mkTable`, and each read costs one unit.

The value lemmas and the bound are machine-checked; that each `tick` and each `pure` covers the work it stands for is not, and belongs to the trusted base. An auditor should read the `ticks` and `pures` of `Timed.lean`, `K3CostLB.lean` and `K3Cost.lean`; each is annotated with what it counts.

Where the fourth power comes from. The proof bounds every counted step in terms of one number $N = n + m + 1$, which exceeds the length of every list the program builds: agents, upgraded agents (with k^*), exposed agents, the chain and the order have at most $n+1$ elements; the remaining goods, the junk and the pools at most m ; the placement list at most $n + m$. Appendix B lists the bound for each step. The degree-four term comes from the upgrade loop: at most n rounds, each testing up to n candidates, each test deciding whether $b_k \in NA$ by scanning n agents with a membership test in U ($36N^4$ in all, `EFX.K3.1bUpC_cost`). Stage L costs at most $310N^4$ (`EFX.K3.1bPlusC_cost`), Stage R at most $15N^2$ per round, and the whole program at most $400N^4$. Section 6.5 bounds the same steps in terms of n and m separately.

6.3 Bit complexity

The unit-cost model charges one unit for arithmetic on numbers of any size. The next corollary shows that this is harmless here.

Corollary 3 (bit complexity; written, not formalized). *Let b be the largest bit length of an input value. If each operation of the cost model costs the bit length of its operands, $K3ALG$ takes $O((n + m)^4(b + \log(n + m)))$ bit operations.*

Proof. We read the counted program, which is in the files `Timed.lean`, `K3CostLB.lean` and `K3Cost.lean`. It forms no products. Values enter only in Stage R and in the computation of the rankings: each is compared with 0 or with another value of the same agent, and rule R1 compares $v_i(p)$ with $v_i(M \setminus \{p\})$, a sum of values of agent i over the remaining goods. So every number derived from values is a sum of at most m values of one agent, below $m2^b$, and Stage L reads no values at all. The other numbers are ranks in $\{0, 1, 2, 3\}$, slot counts and their sums (at most $2n$), list lengths (at most $n + m$) and block counters (at most n), and agents and goods are indices below n and m . So every operand, including table addresses, has $O(b + \log(n + m))$ bits, and each counted operation costs that many bit operations. Multiply by the bound of Theorem 8. $\square$

6.4 Rational and real values

Proof (of Corollary 1). Multiply agent i 's values by a common denominator of them. This multiplies every sum of i 's values by the same positive number, so every comparison between two such sums is preserved, and with it the relevant goods and the EFX_0 condition, as in Lemma 6. So the scaled instance is 3-relevant, K3ALG's output on it is EFX_0 for the scaled values (Theorem 3), and hence for the rational values. $\square$

In Lean: `EFX.K3.algoRat_efx0`, for core Lean's rationals, with `EFX.K3.algoRat`, K3ALG on `EFX.ratScale`, which multiplies agent i 's values by the product of their denominators (`EFX.ratScale_val`); `EFX.ratScale_agree` is the preservation of comparisons, and `EFX.ratScale_efx0_iff` and `EFX.ratScale_numRelevant` its consequences. Every decision of K3ALG is also the same on the scaled values as it would be on the rational values, since each compares two sums of one agent's values; this remark, and the cost of computing the scaling, are written, not formalized.

For real values, the model must say what an algorithm can do with a real number. We use a comparison oracle, in the style of the real RAM.

Corollary 4 (real values, comparison model). *Let every agent positively value at most three goods, with nonnegative values in $\mathbb{R}_{\geq 0}$, or in any linearly ordered cancellative commutative monoid. Suppose an oracle answers at unit cost whether $x \leq y$, where x and y are sums of at most two values of one agent (the same value twice allowed for agents with one or two relevant goods), and adding two values of one agent costs one operation. Then a program computes, with at most $n(m+12)$ oracle calls, natural-number valuations w that agree with v on every comparison of subset sums, as in Lemma 6, and K3ALG on w returns an allocation that is EFX_0 for v . In all it performs at most $n(m+12) + 10nm + 971n + 400(n+m+1)^4$ operations, oracle calls included, and at most $n(m+12) + 10nm + 971n + 145n^4 + 72n^2m + 119n + 22m + 3$.*

Proof. The program inspects the values only through the oracle, and it adds values of one agent. For each agent i it finds R_i , in index order, with m calls $v_i(g) \leq 0$. Let x_1, x_2, x_3 be the values of i 's first three relevant goods; with one relevant good, $x_2 = x_3 = x_1$, and with two, $x_3 = x_1$ (a phantom slot). The program makes six additions and twelve calls: $x_j \leq x_k$ for $j \neq k$, and $x_j \leq x_k + x_l$ and $x_k + x_l \leq x_j$ for $\{j, k, l\} = \{1, 2, 3\}$. These answers determine the ties and the sign of each $x_j - (x_k + x_l)$, which, as in the proof of Lemma 6 (Appendix A), decide every comparison of two

subset sums of the relevant goods. The program sets w_i on these goods to the first of the 31 natural-number triples of the Lean proof of Lemma 6 (all permutations of the representatives of Appendix A) that gives the same answers, and $w_i(g) = 0$ elsewhere, and stores w as an $n \times m$ table. The entries of w are at most 5. With one or two relevant goods, w_i may differ from the representatives listed in Appendix A (for values $(1, 3)$ it is $(1, 3)$) and still agrees with v_i . Per agent this is $m + 12$ oracle calls and $O(m)$ other operations: at most $n(m + 12) + 10nm + 971n$ in all. Then K3ALG on w takes at most $400(n + m + 1)^4$ operations (Theorem 8), and at most $145n^4 + 72n^2m + 119n + 22m + 3$ (Theorem 9), with numbers of $O(\log m)$ bits; its output is EFX₀ for w (Theorem 3), hence for v (Lemma 6). $\square$

In Lean: `EFX.K3.algoOrd_efx0` (correctness, assuming a correct oracle), `EFX.K3.algoOrdC_cost` and `EFX.K3.algoOrdC_cost_fine` (the two operation bounds), with `EFX.K3.surrogateC_cost` (the surrogate's cost; its coefficient of the cost of a call bounds the calls), `EFX.K3.agree_surrogate` (the surrogate agrees with v) and `EFX.K3.algoOrd_eq` (the program is K3ALG on the surrogate). The values may lie in any type of the class `EFX.OrderedValue`, and that $\mathbb{R}_{\geq 0}$ is one is the standard fact of Section 7.2. The written argument needs only comparisons of two subset sums, at most six per agent after the m comparisons with 0 (three to sort, the ties, and the top against the other two; Appendix A); the program's comparisons for agents with one or two relevant goods, such as $v_i(g_2) \leq 2v_i(g_1)$, are not of that kind, which is why the oracle of the corollary compares sums of at most two values with repetition.

Alternatively, K3ALG can run on real inputs directly. Every decision it makes compares two sums of one agent's values (relevance, the favourite good, rule R1, and the three comparisons of the sort), and Lemma 6 answers each such comparison for v as for w , so the run on v is the run on w and returns the same allocation. This remark is written, not formalized.

6.5 A finer bound and measurements

Layer (ii): a finer bound. Bounding each step by its own parameters instead of N , with a bound n on every list of agents ($n+1$ for the upgraded agents after the rotation, k^* included) and m on every list of goods, gives the following (Appendix B, last column). It is proved for the same program and in the same cost model as Theorem 8.

Theorem 9 (finer operation count; `EFX.K3.algoC_cost_fine`). *For every instance with $n \geq 1$ agents and m goods, with any natural-number*

values (no assumption on the number of relevant goods),

$$(\text{algoC I hn}).\text{cost} \leq n^4 + 20n^3 + 25n^2m + 124n^2 + 47nm + 119n + 22m + 3;$$

hence $\leq 145n^4 + 72n^2m + 119n + 22m + 3$ (`EFX.K3.algoC_cost_fine'`)
and $\leq 270(n^4 + n^2m)$ (`EFX.K3.algoC_cost_fine''`).

The dominant terms come from the upgrade loop, at most $n^4 + 17n^3 + n^2m + 12n^2 + 4nm + 2n + m$ (`EFX.K3.lbUpC_cost_fine`): at most n rounds, each testing every agent, each test deciding whether $b_k \in NA$ by scanning the agents with a membership test in U and whether $c_k \in J$. The n^2m terms come from Stage R (n rounds, n agents, an R1 test over the goods), the processing order, Phase 1's picks, the blocks, and the search for two meeting sets π_x in both owner tests (pairs of exposed agents, membership in J); there is no nm^2 or m^2 term. Stage L and Stage R are bounded by `EFX.K3.lbPlusC_cost_fine` and `EFX.K3.reduceC_cost_fine`.

Layer (iii): a faster implementation (written, not formalized). A Python implementation of the same algorithm, the function `fast` of the file `k3/k3algo.py` of the repository, replaces the naive scans by a heap of the agents to which R1 applies (under 3-relevance, R1 keeps applying to an agent once it applies), a heap of the agents with a good missing (for Phase 1's order), per-good counters of the agents that rank a good above their pick (for the upgrades), and an index from junk goods to exposed agents (for `HITSET`). Its input lists, for each agent, the goods it values with their values. Its written analysis, not formalized, gives $O((n + m) \log n)$ operations after the input is read; reading the input takes $O(n + m)$ steps in this form, and a dense $n \times m$ table would take nm .

Measurements. The repository [9] records the following measurements of implementations. None of them is part of a theorem.

- `fast` returns the same allocation as a literal transcription of the Lean definitions on 200,000 random instances; on 675 instances both agree with Lean's own evaluation of `EFX.K3.algo`, every Lean output passes a raw EFX_0 check, and every Lean operation count is below the Lean-checked bound. The largest count divided by $(n + m + 1)^4$ falls from 0.38 at $n = 2$ to 0.002 at $n = 30$, against the factor 400 of the bound.
- Median running times of `fast` (Python 3.11, one CPU, three instances per point), on random instances in which every agent values exactly three goods and is balanced, with $m = 1.5n$ ("core") and $m = 2n$

(“core2m”), and on random instances in which each agent values between zero and three goods (“general”), in seconds:

n	core	core2m	general	literal transcription (core2m)
80	0.0006	0.0007	0.0004	0.30
1,000	0.0073	0.0074	0.0051	–
10,000	0.082	0.089	0.062	–
100,000	1.53	1.58	0.84	–

The log–log slopes of **fast** between consecutive sizes are between 1.0 and 1.4, consistent with $O((n + m) \log n)$.

7 The Machine-Checked Proof

This section describes the Lean 4 formalization [9]: what it covers, what a reader must trust, which axioms it uses, how to read its main statements, and what it leaves out. The further checks that the repository runs on it (a replay checker, an independently written statement, a second proof and a second kernel) are in Appendix D; Appendix D.5 maps every result of this paper to its Lean names, and Appendix E maps every object of Section 4 to its Lean definition.

7.1 What is formalized

- *Theorems 1 and 2*, for natural-number values (`EFX.target`, `EFX.LB.corollaryD`) and for values in any linearly ordered cancellative commutative monoid (`EFX.target_ordered`, `EFX.corollaryD_ordered`).
- *Theorem 3*: the algorithm, a computable function (`EFX.K3.algo`); its correctness (`EFX.K3.algo_efx0`); and its operation bounds, for the counted program whose value is the algorithm (`EFX.K3.algoC_cost`, and the finer Theorem 9, `EFX.K3.algoC_cost_fine`).
- *Section 5* in full: the invariants of Phase 1, the validity of the upgrades, soundness, the owner criterion, Lemmas 10–13 and the remark after Lemma 10 that r lies in the last block, Theorems 5, 6 and 7, Propositions 1 and 2, and the example of Remark 1. LB^+ is formalized both as a relation over all its runs and as a computable function, the one `K3ALG` uses.
- *Section 3*: peeling (rules `R1` and `R2`, and the general Lemma 3); Lemmas 4, 5 and 6; and Corollary 2, for natural-number values, for every run of serial dictatorship. Also the reduction to cores of Section 8.

- *Corollaries 1 and 4*: K3ALG on nonnegative rationals, scaled to natural numbers (`EFX.K3.algoRat_efx0`), and on nonnegative values in any linearly ordered cancellative commutative monoid, in the comparison model, with its operation count (`EFX.K3.algoOrd_efx0`, `EFX.K3.algoOrdC_cost`).
- *Examples* (Appendix C), checked by `decide`: four runs of K3ALG, among them a peel followed by the owner r , a rotation with the owner k^* , and the run of Remark 1; a run of K3ALG on integer values through the oracle of Corollary 4; and a run of serial dictatorship.

7.2 The trusted base

To trust Theorems 1–3 one must read, besides the statements themselves, only the definitions they use. For natural-number values these are six definitions in `lean/EFX/Model.lean`, copied from [10] with the namespace renamed:

```
def finSum : (k : Nat) → (Fin k → Nat) → Nat
| 0, _ => 0
| k+1, f => finSum k (fun i => f i.castSucc) + f (Fin.last k)
structure Inst where
  n : Nat
  m : Nat
  v : Fin n → Fin m → Nat
abbrev Alloc := Fin I.m → Fin I.n
def bundleVal (X : I.Alloc) (i j : Fin I.n)
  (ex : Option (Fin I.m)) : Nat :=
  finSum I.m (fun g =>
    if X g = j ∧ ex ≠ some g then I.v i g else 0)
def EFX0 (X : I.Alloc) : Prop :=
  ∀ i j : Fin I.n, i ≠ j → ∀ g : Fin I.m, X g = j →
    I.bundleVal X i j (some g) ≤ I.bundleVal X i i none
def numRelevant (I : Inst) (i : Fin I.n) : Nat :=
  finSum I.m (fun g => if 0 < I.v i g then 1 else 0)
```

In words: `finSum` is the sum over `Fin k`; an instance has n agents, m goods and values $v_i(g) \in \mathbb{N}$; an allocation assigns every good to one agent, so completeness and disjointness are built in; `bundleVal X i j ex` is agent i 's value for j 's bundle, without the good `ex` if one is given; `EFX0` is the definition of Section 1.1, for every good of X_j , including goods that i values at zero; and `numRelevant` is $|R_i|$.

For values in an ordered type, the trusted base is the class `EFX.OrderedValue` and a mirror of the model above in which `Nat` is replaced by the value type (`0Inst`, `finSum0`, and `0Inst.bundleVal`, `0Inst.EFX0`, `0Inst.numRelevant`, word for word). The class requires `0`, `+` and `≤` such

that $+$ is associative and commutative with identity 0, $\leq$ is a total order, and $a \leq b \iff a + c \leq b + c$: the axioms of a linearly ordered cancellative commutative monoid. Nonnegativity of values is a hypothesis of the theorems, a good is relevant when $v_i(g) \leq 0$ fails, and balance reads $v_i(g) + v_i(g) \leq v_i(M)$. At $\mathbb{N}$ the mirrored model agrees with `Model.lean`, and the specializations of the ordered theorems have exactly the types of `EFX.target` and `EFX.LB.corollaryD` (`EFX.target_of_ordered`, `EFX.corollaryD_of_ordered`). That $\mathbb{R}_{\geq 0}$ satisfies the axioms is a standard fact; core Lean has no real numbers, so this last step is not a Lean theorem. Thus Theorems 1 and 2 are Lean-checked for this abstract ordered additive structure, and their instance at $\mathbb{R}_{\geq 0}$ rests on that standard fact. Corollary 4 is stated over the same mirrored model, and Corollary 1 over its instance at core Lean’s rationals `Rat`, for which the library proves the axioms.

For the running time, the trusted base also contains the cost model: that every `tick` and `pure` of the counted program covers the work it stands for (Section 6.2).

7.3 Axioms and checks

The formalization uses core Lean 4 (v4.34.0), with no packages. The word `sorry` appears nowhere, and the build is free of warnings. Every declaration of the library depends only on the three standard axioms `propext`, `Classical.choice` and `Quot.sound`; this excludes unfinished proofs, `native_decide` (which would trust the compiler) and new axioms. The repository’s check script verifies this for all declarations on every pull request, and every main result also carries a `#print axioms` certificate at the end of its file (Appendix D lists the script’s further checks).

7.4 How to read the main statements

```
theorem EFX.target (I : Inst) (hn : 0 < I.n)
  (h : ∀ i, numRelevant I i ≤ 3) : ∃ X : I.Alloc, I.EFX0 X
```

This is Theorem 1 for natural-number values. The hypothesis `hn` asks for at least one agent: with no agents and some goods there is no allocation at all, since no function from goods to agents exists. The hypothesis `h` is 3-relevance.

```
theorem EFX.LB.corollaryD (I : Inst) (hn : 0 < I.n)
  (h3 : ∀ i, numRelevant I i = 3)
  (hbal : ∀ i g, 2 * I.v i g ≤ finSum I.m (I.v i)) :
  ∃ X : I.Alloc, I.EFX0 X ∧ ∃ o, ∀ j, j ≠ o →
    finSum I.m (fun g => if X g = j then 1 else 0) ≤ 2
```

This is Theorem 2. `h3` says that every agent values exactly three goods, and `hbal` that no good is worth more than all of the agent's goods together minus itself, which for three relevant goods is balance, $v_i(a_i) \leq v_i(b_i) + v_i(c_i)$. The conclusion counts the goods of each bundle: every agent other than o has at most two.

```
def EFX.K3.algo (I : Inst) (hn : 0 < I.n) : I.Alloc :=
  (algoC I hn).val
theorem EFX.K3.algo_efx0 (I : Inst) (hn : 0 < I.n)
  (h : ∀ i, numRelevant I i ≤ 3) : I.EFX0 (algo I hn)
theorem EFX.K3.algoC_cost (I : Inst) (hn : 0 < I.n) :
  (algoC I hn).cost ≤ 400 * (I.n + I.m + 1) ^ 4
```

These are Theorem 3. The algorithm takes the proof `hn` as an argument, for the reason just given. `algo` is a computable function (Lean's `#eval` runs it), and the bound holds for every instance, whatever the number of relevant goods.

```
theorem EFX.target_ordered {V : Type} [OrderedValue V] (I : 0Inst V)
  (hn : 0 < I.n) (hv : ∀ i g, 0 ≤ I.v i g)
  (h : ∀ i, I.numRelevant i ≤ 3) : ∃ X : I.Alloc, I.EFX0 X
theorem EFX.corollaryD_ordered {V : Type} [OrderedValue V] (I : 0Inst V)
  (hn : 0 < I.n) (hv : ∀ i g, 0 ≤ I.v i g)
  (h3 : ∀ i, I.numRelevant i = 3)
  (hbal : ∀ i g, I.v i g + I.v i g ≤ finSum0 I.m (I.v i)) :
  ∃ X : I.Alloc, I.EFX0 X ∧ ∃ o, ∀ j, j ≠ o →
    finSum I.m (fun g => if X g = j then 1 else 0) ≤ 2
```

These are Theorems 1 and 2 over the ordered type V , derived from the statements over $\mathbb{N}$ by Lemma 6 (`EFX.l12`, `EFX.efx0_iff_of_agree`). The hypothesis `hv` is nonnegativity.

```
theorem EFX.K3.algoC_cost_fine'' (I : Inst) (hn : 0 < I.n) :
  (algoC I hn).cost ≤ 270 * (I.n ^ 4 + I.n ^ 2 * I.m)
theorem EFX.K3.algoRat_efx0 (I : 0Inst Rat) (hn : 0 < I.n)
  (hv : ∀ i g, 0 ≤ I.v i g)
  (h : ∀ i, I.numRelevant i ≤ 3) : I.EFX0 (algoRat I hn)
theorem EFX.K3.algoOrd_efx0 {V : Type} [OrderedValue V]
  {le : V → V → Bool} (hle : ∀ x y, le x y = true ↔ x ≤ y)
  (I : 0Inst V) (hn : 0 < I.n) (hv : ∀ i g, 0 ≤ I.v i g)
  (h : ∀ i, I.numRelevant i ≤ 3) : I.EFX0 (algoOrd le I hn)
theorem EFX.K3.algoOrdC_cost {V : Type} [OrderedValue V]
  (le : V → V → Bool) (I : 0Inst V) (hn : 0 < I.n) :
  (algoOrdC le I hn).cost ≤ I.n * (I.m + 12) + 10 * (I.n * I.m)
  + 971 * I.n + 400 * (I.n + I.m + 1) ^ 4
theorem EFX.Inst.sdrun_efx0 (I : Inst) (h : ∀ i, numRelevant I i ≤ 2)
  {order : List (Fin I.n)} (hord : order.Nodup) (hall : ∀ i, i ∈ order)
  {X : I.Alloc} (hX : SDRun I.v order (List.finRange I.m) X) : I.EFX0 X
```

These are Theorem 9, Corollaries 1 and 4, and Corollary 2. Their statements use definitions beyond the trusted base, which a reader must compare with the text: `EFX.K3.algoRat` is `EFX.K3.algo` on `EFX.ratScale`, the instance whose values are agent i 's values times the product of their denominators, as natural numbers; `EFX.K3.algoOrd` is the value of the counted program `EFX.K3.algoOrdC`, which receives the oracle `le`, computes the surrogate `EFX.K3.surrogate` and runs `EFX.K3.algoC` on it, and `hle` says that the oracle is correct; and `EFX.SDRun` is the set of runs of serial dictatorship in the order `order` (each agent once, by `hord` and `hall`), with any favourite at each step.

Intermediate statements. The Lean statements of Section 5's results are stated over the formalization's own definitions of the state of LB^+ (picks, blocks, junk, slots, exposure, and so on, in `PreAlloc.lean`, `Blocks.lean`, `OwnerR.lean`, `Rotation.lean` and `LBPlus.lean`). To know that such a statement is the theorem of this paper, a reader must compare those definitions with Section 4; Appendix E lists, for each object of Section 4, the Lean definition that formalizes it. This is not needed to trust Theorems 1–3, whose statements use only the trusted base. For instance, `EFX.LB.lbPlusRun_sound` quantifies over all runs of LB^+ : every order of Phase 1 with R1 priority, every upgrade order, every need chain and every completion satisfying (OC).

7.5 What is not formalized

- Corollary 3, the bit complexity, and the analysis of the faster implementation `fast` (Section 6.5).
- The remarks of Section 6.4 that K3ALG run directly on the rational or real values makes the same decisions as on the scaled values or on w , and the cost of computing the scaling of Corollary 1.
- Corollary 2 for values other than natural numbers (its form for an ordered type).
- Proposition 4, the limits of the shape, and Lemma 1.
- That $\mathbb{R}_{\geq 0}$ satisfies the axioms of `EFX.OrderedValue` (core Lean has no reals), and that each `tick` of the counted program covers its work.
- Lemma 2 and the graph-theoretic facts about cores used in Section 8; the computations of Section 8 are checked by separate programs, not by Lean.

8 Computational Certification and the Limits of the Shape

This section explains what exhaustive computation over small instances establishes, independently of the proof of Section 5, and shows, by small counterexamples, that the shape of Theorem 2 cannot be improved in the obvious ways. The sizes of the runs, the tests, the certificate format and how to reproduce the computations are in Appendix F.

8.1 Cores

Definition 10 (core). *A core is an instance with at least two agents in which (K1) every agent values exactly three goods, (K2) every agent is strictly balanced, (K3) every agent has at most one private good (a good that no other agent values), and (K4) every good is valued by some agent.*

Cores are the instances that peeling and the removal of unvalued goods cannot simplify. *Rule R2* is the case of Lemma 3 in which P is the set of agent i 's private goods, $|P| \geq 2$ and $v_i(P) \geq v_i(M \setminus P)$.

Proposition 3 (reduction to cores). *If every core with at most N_0 agents has an EFX₀ allocation, then so does every 3-relevant instance with between 1 and N_0 agents.*

Proof. Induction on $n+m$ over 3-relevant instances with $1 \leq n \leq N_0$. One agent takes everything. Otherwise: (1) if some good is valued by nobody, use Lemma 5; (2) else if rule R1 applies to some agent, use Lemma 3; (3) else every agent values exactly three goods and is strictly balanced (Lemma 4); if some agent i has at least two private goods, let P be all of them: if $|P| = 3$, condition (i) of Lemma 3 is trivial, and if $P = \{x, y\}$ and $R_i \setminus P = \{z\}$, then $v_i(x) + v_i(y) \geq v_i(b_i) + v_i(c_i) > v_i(a_i) \geq v_i(z)$, so Lemma 3 applies (rule R2); (4) else the instance is a core with at most N_0 agents, covered by the hypothesis. Each of (1)–(3) passes to a smaller 3-relevant instance with at least one agent. $\square$

In Lean: `EFX.core_reduction`. Two more facts make the certification finite.

- *Components.* The *incidence graph* of an instance joins agent i to good g when $g \in R_i$. Agents in different connected components value nothing in each other's bundles, so EFX₀ allocations of the components of a core combine into one of the core, and each component is itself a core. So it suffices to certify *connected* cores.

- *Rankings suffice.* By Lemma 2(ii), in a core with strict rankings EFX₀ depends only on the rankings: 6 possible rankings per agent, 6^n *ranking profiles* per core. By Lemma 2(iii), an allocation that is EFX₀ for a strict profile consistent with tied values is EFX₀ for the tied values too. So certifying every strict profile covers all values.

Counting incidences, a core has $m \leq 2n$ goods. Cores with $m = 2n$ are solved directly: an agent holding two of its own three goods is safe, by balance (`EFX.Inst.safe_of_two_own`), and in a connected core with $m = 2n$ every agent can be given its private good and one shared good (this second fact is written, not formalized: its proof is recorded in the repository).

8.2 What the computation establishes

For every number of agents $n \leq 7$ and every number of goods m , the connected cores were listed up to isomorphism, and for every core and every one of its 6^n strict ranking profiles an EFX₀ allocation was found and stored. A separately written checker re-checks the stored allocations from the raw EFX₀ definition, and the completeness of each list of cores was checked by a second, independent method. Appendix F gives the sizes of the runs, the methods, the certificate format and how to reproduce them. With Proposition 3, the components argument and the cores with $m = 2n$, the computation establishes the following.

- *Theorem 1 for $n \leq 7$* , verified by exhaustive computation independently of Section 5: neither the search nor the checker uses LB⁺ or its proof.
- *Theorem 2 for every core with $n \leq 6$, every connected core with $n = 7$, and every connected core with $n = 8$ and $m \geq 13$* , verified by exhaustive computation in the same way. The shape of Theorem 2 was first observed in these computations.

It establishes nothing for larger n . Some certificates were produced by an earlier construction, LB; what the runs show about LB itself is stated in Appendix F.

8.3 The limits of the shape

Could every core have an EFX₀ allocation with all bundles of at most two goods, or with one bundle of exactly three? No.

Proposition 4 (limits of the shape; theorem, proof in this paper). *Take strict rankings and strictly balanced values.*

- (a) *Let agents 0, 1, 2 rank $g_0 \succ g_1 \succ p_i$, where p_i is private to agent i . No EFX_0 allocation has all bundles of at most two goods, and $\{g_0\}, \{g_1\}, \{p_0, p_1, p_2\}$ (to the three agents in any order) is EFX_0 .*
- (b) *Let agents 0, 1, 2, 3 rank $g_0 \succ g_2 \succ p_0, g_0 \succ g_2 \succ p_1, g_1 \succ g_2 \succ p_2$ and $g_1 \succ g_2 \succ p_3$, with p_i private to agent i . Every EFX_0 allocation with at most one bundle of more than two goods has bundle sizes 4, 1, 1, 1.*

Proof. (a) Five goods in three bundles of at most two goods have sizes 2, 2, 1, so exactly one good is alone. Good g_0 is everyone's top and has one holder, so two agents do not hold their top: case (T) of Lemma 2 fails for them, and cases (C) and (E) need two or three alone goods. So each of these two agents is in case (P) or (B), and both hold g_1 , which is impossible. In $\{g_0\}, \{g_1\}, \{p_0, p_1, p_2\}$, the holder of g_0 is in case (T), the holder of g_1 in case (B), and the third agent in case (C).

(b) Each of the tops g_0 and g_1 has a claimant that does not hold it, a *loser*; the two losers are different agents. A loser is safe only by (P) or (B), which need it to hold g_2 , or by (C) or (E), which need g_2 alone. The two losers cannot both hold g_2 , so g_2 is alone, and then no agent can be in case (P), which would put g_2 with another good. A loser holding $\{g_2\}$ is in case (B) or (E) and needs its top alone; a loser not holding g_2 is in case (C) or (E) and needs its top and g_2 alone. So g_0, g_1 and g_2 are alone, held by three different agents, and with at most one bundle of more than two goods the fourth bundle is $\{p_0, p_1, p_2, p_3\}$. Such an allocation exists: g_0 to agent 0, g_1 to agent 2, g_2 to agent 1 and the private goods to agent 3 is EFX_0 (cases T, B, T, C). $\square$

Exhaustive computation shows that no core with two agents needs a bundle of more than two goods, so (a) is a smallest example. The repository records the following counterexamples, each with a checker [9]:

- bundles of at most two goods suffice in a core: false for $n = 3, m = 5$ (Proposition 4(a); 14 of the 216 ranking profiles of that core fail);
- bundles of at most two goods suffice when $m \leq 2n - 2$: false; smallest $n = 4, m = 6$; at $n = 6, m = 10$, 57 of the 211 connected cores have a failing profile (confirmed by a second, independent encoding);
- the same for instances that are not cores: four agents with identical values 4, 3, 2 on three goods, and three goods valued by nobody;
- one bundle of three goods suffices: false for $n = 4, m = 7$ (Proposition 4(b)); at $n = 6, m = 11$, 3 of the 25 connected cores need four.

9 Conclusion and Open Problems

Every instance in which each agent positively values at most three goods has a complete EFX₀ allocation, and one can be computed in polynomial time by K3ALG: peel agents by rule R1, run a serial dictatorship that serves agents who lost goods first, put the leftover goods into free slots, and give the surplus to one owner, after at most one rotation. When every agent values exactly three goods and is balanced, there is a complete EFX₀ allocation in which all but at most one agent receive at most two goods. The correctness of K3ALG and its operation bounds, in an explicit unit-cost model, are machine-checked on the program itself.

Four relevant goods per agent remain open; there, one rotation of the kind used here is not enough, already for $n = 5$ with a single agent valuing four goods [9]. We also do not determine the least possible size of the large bundle: Proposition 2 gives the size that the construction produces, and Proposition 4(b) shows that three goods do not always suffice.

Acknowledgments. The proofs, code and Lean formalization were developed with the assistance of AI coding agents (Claude Code).

References

1. Afshinmehr, M., Ashuri, A., Mahmoudkhan, P., Mehlhorn, K., Shahrezaei, A.M.: EFX allocations exist on multi-graphs. arXiv preprint arXiv:2606.18665 (2026)
2. Akrami, H., Mayorov, A., Mehlhorn, K., Srinivas, S., Weidenbach, C.: A counterexample to EFX $n \geq 3$ agents, $m \geq n + 5$ items, submodular valuations via SAT-solving. arXiv preprint arXiv:2604.18216 (2026)
3. Alkassar, E., Fouz, M., Mehlhorn, K.: Complete EFX allocations exist for four additive agents and up to nine goods. arXiv preprint arXiv:2608.08590 (2026)
4. Amanatidis, G., Birmpas, G., Filos-Ratsikas, A., Voudouris, A.A.: Fair division of indivisible goods: A survey. In: Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence (IJCAI-22). pp. 5385–5393 (2022). <https://doi.org/10.24963/ijcai.2022/756>, arXiv:2202.07551
5. Chaudhury, B.R., Garg, J., Mehlhorn, K.: EFX exists for three agents. Journal of the ACM **71**(1), 4:1–4:27 (2024). <https://doi.org/10.1145/3616009>, conference version in Proc. 21st ACM Conference on Economics and Computation (EC 2020), pp. 1–19; full version arXiv:2002.05119
6. Christodoulou, G., Fiat, A., Koutsoupias, E., Sgouritsa, A.: Envy-freeness up to any good allocations on graphs. Mathematics of Operations Research (2026). <https://doi.org/10.1287/moor.2025.0915>, conference version: “Fair Allocation in Graphs”, in Proc. 24th ACM Conference on Economics and Computation (EC 2023), pp. 473–488
7. Kaviani, A., Seddighin, M., Shahrezaei, A.: Almost envy-free allocation of indivisible goods: A tale of two valuations. In: Web and Internet Economics (WINE 2024). pp. 261–276. Lecture Notes in Computer Science, Springer (2026). https://doi.org/10.1007/978-3-032-08560-3_15, full version arXiv:2407.05139

8. Lianeas, T., Sgouritsa, A., Sotiriou, M.M.: EFX allocation in (multi)hypergraphs. *Proceedings of the AAAI Conference on Artificial Intelligence* **40**(20), 17102–17110 (2026). <https://doi.org/10.1609/aaai.v40i20.38759>, AAAI-26; arXiv:2608.03171
9. Lin, E.: efx0-three-goods: EFX₀ with at most three relevant goods per agent (proofs, certificates, Lean formalization). GitHub repository, commit `c6d1871`, <https://github.com/evanlin23/efx0-three-goods> (2026)
10. Lin, E., Osterhaus, G.: Strong EFX allocations for 2-relevant agents: Lean 4 formalization. GitHub repository `mrd-efx`, tag `v1.2.0` (commit `fa42aa1`, 23 September 2026), <https://github.com/evanlin23/mrd-efx> (2026)
11. Mackenzie, S., Suzuki, M.: Counterexamples to EFX for submodular and subadditive valuations. arXiv preprint arXiv:2605.06451 (2026)
12. Mahara, R.: Extension of additive valuations to general valuations on the existence of EFX. *Mathematics of Operations Research* **49**(2), 1263–1277 (2024). <https://doi.org/10.1287/moor.2022.0044>, conference version in *Proc. 29th European Symposium on Algorithms (ESA 2021)*, *LIPICs* 204, 66:1–66:15; arXiv:2107.09901
13. Sgouritsa, A., Sotiriou, M.M.: On the existence of EFX allocations in multigraphs. arXiv preprint arXiv:2502.09777 (2025), extended abstract, without the bipartite case, in *Proc. 24th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2025)*, pp. 2735–2737
14. Viswanathan, M., Mehta, R.: On the existence of EFX under picky or non-differentiative agents. In: *Proceedings of the 23rd International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2024)*. pp. 2534–2536 (2024). <https://doi.org/10.65109/osos1146>, extended abstract

A Proof of Lemma 6

Proof (of Lemma 6). Fix an agent i ; w_i is built from v_i alone. Goods outside R_i contribute 0 to both sides of a comparison under v_i , and will contribute 0 under w_i , so S and T may be replaced by $S \cap R_i$ and $T \cap R_i$. By additivity, removing the goods common to S and T changes neither comparison. So it suffices that w_i preserves every comparison $v_i(X) \leq v_i(Y)$ between disjoint subsets X, Y of R_i .

Name the relevant goods so that their values are $a \geq b \geq c > 0$ (fewer letters if $|R_i| < 3$). The comparisons between disjoint $X, Y \subseteq R_i$ are of three kinds: with an empty side, decided by positivity, since a nonempty set of relevant goods has positive value; between two single goods, decided by the ties among a, b, c ; and between a single good and the other two, where $b < a + c$ and $c < a + b$ always hold, and only a against $b + c$ is free. Two disjoint pairs do not fit in three goods. So every comparison is determined by which of $a = b$ and $b = c$ hold and by the sign of $a - (b + c)$. Choose w_i on R_i , in the same order, by the following representatives, and $w_i(g) = 0$ for $g \notin R_i$:

pattern	w_i	pattern	w_i
$a > b > c, a < b + c$	(4, 3, 2)	$a > b = c, a < 2b$	(3, 2, 2)
$a > b > c, a = b + c$	(3, 2, 1)	$a > b = c, a = 2b$	(2, 1, 1)
$a > b > c, a > b + c$	(5, 2, 1)	$a > b = c, a > 2b$	(3, 1, 1)
$a = b > c$ (so $a < b + c$)	(2, 2, 1)	$a = b = c$	(1, 1, 1)
two relevant goods, $a > b$	(2, 1)	two relevant goods, $a = b$	(1, 1)
one relevant good	(1)		

Each representative has the ties of its pattern, and $a - (b + c)$ has the stated sign: $4 < 5$, $3 = 3$, $5 > 3$, $2 < 3$, $3 < 4$, $2 = 2$, $3 > 2$ and $1 < 2$. So w_i and v_i agree on every comparison between disjoint subsets of R_i , hence on every comparison $v_i(S) \leq v_i(T)$.

EFX₀ is a conjunction of comparisons $v_i(X_j \setminus \{h\}) \leq v_i(X_i)$, so an allocation is EFX₀ for v if and only if it is for w . Relevance is the comparison of $\{g\}$ with $\emptyset$, and balance is the comparison $v_i(\{g\}) \leq v_i(M \setminus \{g\})$ for every good g , so both are preserved. $\square$

In Lean, `EFX.112` proves this for values in any `EFX.OrderedValue`, using all 31 permutations of these representatives, checked by `decide`, instead of sorting; agents with one or two relevant goods are treated as three slots, one or two of them phantom. The choice of a pattern needs at most three comparisons to sort a, b, c , two for the ties and one of a with $b + c$; the formalized program of Corollary 4 asks twelve comparisons instead of sorting (Section 6.4).

B The Cost of Each Step

The machine-checked bound of Theorem 8, step by step, with $N = n + m + 1$; the proof raises each term to a multiple of N^4 and sums. The last column is the order of each step in the finer count of Theorem 9, which Lean also checks stage by stage (`K3CostFine.lean`), with a bound n on the lists of agents and m on the lists of goods.

Step	Lean bound	finer (Thm. 9)
Stage R: at most n rounds, each testing R1 for every agent (favourite, removal, value of the rest)	$15N^2$ per round	$O(n^2m)$ in all
rankings (a table over the agents: relevant goods, sort)	$14N^2$	$O(nm)$
Phase 1's order: at most n steps, each scanning the agents for one with at most two goods left	$15N^3$	$O(n^2m)$
Phase 1's picks (a table; each entry replays Phase 1 up to the agent)	$17N^3$	$O(n^2m)$
upgrades: at most n rounds $\times$ n agents $\times$ ($b_k \in NA$: n agents, membership in U ; $c_k \in J$)	$36N^4$	$n^4 + 17n^3 + n^2m + O(n^2 + nm)$
junk, slots (a table: n agents, each an NA test), sums	$8N^2 + 21N^3 + \dots$	$O(nm + n^3)$
owner test: exposed agents, the meeting pair (pairs of E , membership in J), <code>HITSET</code>	$20N^2 + 24N^3$	$O(n^2m)$
blocks (a table), k^* , need chain ($\leq n$ agents, a frozen test each), rotated picks	$31N^3 + 34N^3 + \dots$	$O(n^2m + n^3)$
rotated state, second owner test (the meeting pair again), completion ($\leq m$ goods: picker, upgraded, placement)	$21N^3 + 24N^3 + 19N^2 + \dots$	$O(n^2m + n^3)$
output table (m goods, each looked up among $\leq n$ peeled goods)	$4N^2$	$O(nm)$
total	$400N^4$	$270(n^4 + n^2m)$

C Verification of the Examples

Examples 2 and 3. The repository contains a script, `paper/k3/examples/trace_examples.py`, that recomputes every intermediate state stated in Section 4.8. It uses the methods of the literal transcription of the Lean definitions, `mirror` in the file `k3/k3algo.py`, in the order in which the Lean program calls them, and prints the states; its output is the file `trace_output.txt`. For each example it then checks that the allocation assembled from the trace equals the outputs of `mirror` and of the independent implementation `fast`, that it is EFX_0 by two brute-force checks

of the definition (every agent i , every other bundle X_j and every removed good $h \in X_j$), and the side claims made in the text: the EFX₀ violation (and EFX) of the wrong slot filling in Example 2; that every completion with owner $r = 3$ in Example 3 violates EFX₀; the cases of Lemma 2 listed in Tables 6 and 8; Example 1; Remark 1; and $\omega = m - 2n + |NA|$ before and after the rotation. All checks pass.

A second file, `paper/k3/examples/lean_examples.lean`, evaluates the Lean program `EFX.K3.algo` with `#eval` on the instances of Examples 2 and 3 and of Remark 1, and checks the outputs of the two examples by `decide`. Lean returns the following owners of the goods $g_0, g_1, \dots$, which are the allocations stated in the text, with the operation counts of `EFX.K3.algoC`:

instance	owners of $g_0, g_1, \dots$	count
Example 2	1, 2, 0, 2, 3, 4, 1, 1, 3	2,407
Example 3	3, 1, 0, 1, 2, 0, 1, 3	2,659
Remark 1	0, 0, 2, 1, 3, 2, 3, 3, 3	1,779

The examples in the Lean library. The library itself checks three runs of K3ALG by `decide`, carried over to `EFX.K3.algo` by `EFX.K3.algo_eq_spec`. In `EFX.K3.Examples.peelOwner_algo`, agent 0 values g_2, g_4, g_5 at 84, 73, 54, agent 1 values only g_1 , at 87, and agent 2 values g_4, g_2, g_3 at 92, 83, 44. Agent 1 is peeled with g_1 ; agents 0 and 2 form one block, picking g_2 and g_4 ; the junk $\{g_0, g_3, g_5\}$ exceeds the two slots; the owner is $r = 2$, agent 0 is exposed with $\pi_0 = \{g_5\}$, and the output is $X_0 = \{g_2, g_5\}$, $X_1 = \{g_1\}$, $X_2 = \{g_4, g_0, g_3\}$. In `EFX.K3.Examples.rotOwner_algo`, agent 0 values g_1, g_2, g_0 at 5, 4, 2, and agents 1 and 2 both value g_1, g_3, g_2 at 3, 3, 2 (so $a = g_1$, $b = g_3$, $c = g_2$, the tie broken by index). Agents 0, 1, 2 form one block and pick g_1, g_3, g_2 ; agents 0 and 1 are frozen; $r = 2$ fails, since agent 0 is exposed and no slot is free for the others; the rotation along $0 \rightarrow 1 \rightarrow 2$ gives agent 0 the goods g_2 and g_0 and makes it the owner, and the output is $X_0 = \{g_0, g_2, g_5\}$, $X_1 = \{g_1\}$, $X_2 = \{g_3, g_4\}$. The trace script recomputes these runs as well. The library also checks the run of Remark 1 (`EFX.K3.Examples.repeatedGood_state`: $\text{HITSET}(E_3) = (g_0, g_0)$; `EFX.K3.Examples.repeatedGood_algo`: the owner receives $4 = \omega + 3$ goods and the output is EFX₀); the instance of `EFX.K3.Examples.peelOwner_algo` with integer values, run through the oracle of Corollary 4 (`EFX.K3.Examples.peelOwnerZ_algoOrd`: agent 0's surrogate values are 4, 3, 2, and the output is the same); and a run of serial dictatorship on a 2-relevant instance (`EFX.K3.Examples.twoRel_serialDict`).

D The Formalization: Audit Layers and Lean Names

This appendix collects the checks that the repository runs on the formalization beyond those of Section 7, and the Lean names of every result.

D.1 Checks run by the repository

The repository’s check script, `lean/check.sh`, runs on every pull request. Besides the axiom check of Section 7.3, it requires a build free of warnings and of `sorry`, refuses debug options, metaprogramming and unsafe code, and replays the whole library in Lean’s replay checker, `leanchecker` with the option `--fresh`, which re-checks every declaration in a fresh kernel.

D.2 An independent statement and a second proof

The repository also contains an independently written statement of Theorems 1 and 2 (`Audit.target_audit`, `Audit.corollaryD_audit`). It represents bundles as lists that partition the goods, sums values by a hand-written recursion, and removes a good with `List.erase`; its definitions were committed before the model was read, and it is derived from `EFX.target` and `EFX.LB.corollaryD`. A second, independent Lean proof of Theorems 1 and 2, through Pareto-maximal pre-allocations instead of LB^+ , is `EFX.C4min.target_K3` and `EFX.C4min.corollaryD_K3` (the latter for cores, Section 8.1).

D.3 A second kernel

At commit `bf81efb` of [9], the kernel `nanoda 0.4.19`, written in Rust and independent of Lean’s C++ kernel, re-checked the export produced by `lean4export` of all 2,934 named declarations of the library together with their dependencies, 8,762 declarations in all, with no error, admitting only the three standard axioms. The library at that commit contains everything formalized in this paper: Theorems 1 and 2, `K3ALG` with its cost bounds, the transfer to ordered values, the corollaries and the independent statement. The negative controls, a `native_decide` proof and a proof of $1 = 2$ that bypasses the elaborator’s kernel check, were rejected by `leanchecker --fresh` and by `nanoda`; the log is `results/audit_kernel_2026-09-27.log`. An earlier run, on commit `ebf27a2`, had covered the natural-number development of Theorems 1 and 2 (726 named declarations).

D.4 Size

The 25 Lean files that Theorems 1–3, their proofs, the independent statement and the examples use have about 9,700 lines.

D.5 Lean names

All names are in `lean/EFX/`; the file is in parentheses. Every declaration of the library depends only on `propext`, `Classical.choice` and `Quot.sound`, which the repository’s check script verifies for all declarations; the main results also carry `#print axioms` certificates.

Result	Lean names (file)
Thm. 1	<code>EFX.target</code> (<code>Target</code>); <code>EFX.target_ordered</code> (<code>RealValues</code>)
Thm. 2	<code>EFX.LB.corollaryD</code> (<code>CorollaryD</code>); <code>EFX.corollaryD_ordered</code> (<code>RealValues</code>)
Thm. 3, correctness	<code>EFX.K3.algo_efx0</code> , <code>EFX.K3.algo_eq_spec</code> (<code>K3Cost</code>); <code>EFX.K3.algoSpec_efx0</code> , <code>EFX.K3.reduce_sound</code> , <code>EFX.K3.lbStage_sound</code> (<code>K3Algo</code>)
Thm. 3, operations (Thm. 8)	<code>EFX.K3.algoC_cost</code> , <code>EFX.K3.algoC_cost'</code> , <code>EFX.K3.lbPlusC_cost</code> , <code>EFX.K3.reduceC_cost</code> (<code>K3CostBound</code>); <code>EFX.Timed.mkTable_cost</code> (<code>Timed</code>)
Thm. 9	<code>EFX.K3.algoC_cost_fine</code> , <code>EFX.K3.algoC_cost_fine'</code> , <code>EFX.K3.algoC_cost_fine''</code> , <code>EFX.K3.lbUpC_cost_fine</code> , <code>EFX.K3.lbPlusC_cost_fine</code> , <code>EFX.K3.reduceC_cost_fine</code> (<code>K3CostFine</code>)
Cor. 1	<code>EFX.K3.algoRat_efx0</code> , <code>EFX.ratScale_val</code> , <code>EFX.ratScale_agree</code> , <code>EFX.ratScale_numRelevant</code> , <code>EFX.ratScale_efx0_iff</code> (<code>K3Extras</code>)
Cor. 4	<code>EFX.K3.algoOrd_efx0</code> , <code>EFX.K3.algoOrd_eq</code> , <code>EFX.K3.agree_surrogate</code> , <code>EFX.K3.surrogateC_cost</code> , <code>EFX.K3.algoOrdC_cost</code> (<code>K3Real</code>); <code>EFX.K3.algoOrdC_cost_fine</code> (<code>K3CostFine</code>)
Lem. 3, rule R1, R2	<code>EFX.peel</code> (<code>Peeling</code>); <code>EFX.peelBundle</code> , <code>EFX.peelEmpty</code> , <code>EFX.peelR2</code> (<code>PeelingR2</code>)
Cor. 2	<code>EFX.Inst.sdRun_efx0</code> , <code>EFX.Inst.serialDict_efx0</code> , <code>EFX.sdRun_efx0</code> , <code>EFX.serialDict_run</code> (<code>K3Extras</code>); <code>existence</code> : <code>EFX.exists_efx0_of_count</code> (<code>Bridge</code>)
Lem. 4	<code>EFX.not_R1</code> (<code>Target</code>); <code>EFX.K3.r1Step_eq_none</code> (<code>K3Algo</code>)
Lem. 5	<code>EFX.Inst.exists_efx0_of_junk</code> (<code>Junk</code>)
Lem. 6	<code>EFX.l12</code> , <code>EFX.efx0_iff_of_agree</code> (<code>RealValues</code>)

Result	Lean names (file)
Lem. 7	EFX.LB.phase1_spec (LBRun); EFX.LB.phase1_run (Blocks)
Lem. 8	EFX.LB.upFinal_valid (Blocks)
Thm. 4	EFX.LB.Valid.sound (PreAlloc)
Lem. 9	EFX.LB.complete_some, EFX.LB.complete_none (PreAlloc)
Lem. 10	EFX.LB.lastOut_exists, EFX.LB.lastOut_terminal (OwnerR)
r in the last block (Sect. 5.5)	EFX.LB.lastOut_lastBlock, EFX.LB.blk_le_lastOut (K3Extras)
Lem. 11, 12	EFX.LB.exposed_lead, EFX.LB.chainEnd_spec (OwnerR)
Lem. 13, Prop. 1	EFX.LB.hitSet_fits, EFX.LB.validOwner_iff, EFX.LB.hitSet_length (OwnerR)
Thm. 5	EFX.LB.theoremA_invalid, EFX.LB.theoremA (OwnerR)
Thm. 6	EFX.LB.theoremB (Rotation)
Thm. 7	EFX.LB.lbPlusRun_sound, EFX.LB.lbPlusOut_exists, EFX.LB.lbPlusOut_exists_chain, EFX.LB.lbPlus_sound (LBPlus)
Prop. 2	EFX.LB.largeBundle_size, EFX.LB.omega_eq, EFX.LB.numFrozen_eq_numNA, EFX.LB.Completion.owner_length_ge, EFX.LB.Completion.owner_length_eq, EFX.LB.BadCase.omega_le (K3Extras)
Rem. 1	EFX.LB.complete_owner_length, EFX.K3.Examples.repeatedGood_state, EFX.K3.Examples.repeatedGood_algo (K3Extras)
Prop. 3	EFX.core_reduction (Target)
two own goods (Sect. 8.1)	EFX.Inst.safe_of_two_own (TwoOwnGoods)
Examples (App. C)	EFX.K3.Examples.peelOwner_algo, EFX.K3.Examples.rotOwner_algo, EFX.K3.Examples.rot_algo (K3Examples); EFX.K3.Examples.peelOwnerZ_algoOrd (K3Real); EFX.K3.Examples.twoRel_serialDict (K3Extras)
Independent statement	Audit.target_audit, Audit.corollaryD_audit (Audit)
Second proof	EFX.C4min.target_K3, EFX.C4min.corollaryD_K3 (K3Theorem)

E Specification Correspondence

The Lean statements of Section 5’s results use the formalization’s own definitions of the state of Stage L. Tables 9 and 10 map each object of Section 4, and of the corollaries, to the Lean definition that formalizes it, so that a reader can compare them. Names are in the namespace `EFX.LB` unless they are written in full, starting with `EFX.`, and files are in `lean/EFX/`. The Lean state is a ranking profile P (the goods a_i , b_i , c_i of each agent), the lists of agents and goods, the picks Y , a function

Table 9. Objects of Section 4 and their Lean definitions.

Object	Lean definition	File
rankings	Profile (fields a , b , c), Profile.rank ;	LBSound,
$a_i \succ_i b_i \succ_i c_i$	computed by EFX.K3.profileOf	K3Algo
Stage R, rule R1	EFX.K3.r1Step , EFX.K3.findR1 , EFX.K3.reduce	K3Algo
R1 priority	R1Prio (a property of an order); K3ALG’s order r1Order	Blocks, CorollaryD
picks Y	phase1 (Phase 1 in a given order)	LBRun
blocks, leaders	blkAux (block number, the insertion steps up to the agent’s turn), leadB (leader test); what the proofs use of them: Run	Blocks
upgrades, U	upgrades (the loop), canUp (its test), lbUp (K3ALG’s U), UpFinal (any order)	LBRun, Blocks
junk J	junkList ; J_0 : junk0	PreAlloc, Blocks
needs N_i	no definition of the set; “ i needs g ” is Profile.Prefers	LBSound
NA	Profile.NA (a proposition), naB (a Boolean)	LBSound, LBRun
frozen	frozenB	LBRun
terminal	IsTerm	OwnerR
cap, S	cap , slotSum ; the slots of all agents but the owner: slotsExcept	LBRun, PreAlloc
ω , $ NA $, $ F $	ω is the length of junkList minus slotSum , as an integer; numNA (NA counted over the goods), numFrozen	K3Extras
base	InBase (membership in the base)	PreAlloc
valid; (UT)	Valid ; the field ut of State	PreAlloc, OwnerR

from agents to optional goods, and the list **up** of upgraded agents; most objects below are functions of this state. Where no single Lean definition corresponds to an object, the table says so.

F Certification: Runs, Tests and Reproducibility

This appendix gives the details behind Section 8: the sizes of the runs and the methods, the tests of LB^+ and K3ALG, and what is needed to reproduce them.

F.1 Runs and methods

For each number of agents n and goods m , the connected cores were enumerated up to isomorphism, and the completeness of each list was checked

Table 10. Table 9, continued; the last three rows are objects of Corollaries 2, 1 and 4.

Object	Lean definition	File
completion, (OC)	<code>Completion</code> (every completion; its field <code>oc</code> is (OC)); K3ALG's completion <code>complete</code> , which places the junk by <code>fill</code>	<code>PreAlloc</code> , <code>LBRun</code>
owner o	no single definition: the argument <code>o</code> of <code>Completion</code> and <code>complete</code> (an optional agent)	<code>PreAlloc</code>
exposed E_o	<code>Exposed</code> , <code>exposedL</code>	<code>OwnerR</code>
π_x	no single definition: one of its goods is <code>pickOne</code> , and two meeting sets are found by <code>meet</code>	<code>OwnerR</code>
HITSET	<code>hitSet</code>	<code>OwnerR</code>
valid owner	<code>ValidOwner</code> , with <code>OwnerOK</code>	<code>OwnerR</code>
candidate r ; k^*	<code>lastOut</code> ; <code>kstar</code>	<code>OwnerR</code>
need chain	<code>IsChain</code> , <code>NeedChain</code> ; K3ALG's scan: <code>chainFrom</code> with <code>isNext</code>	<code>OwnerR</code> , <code>LBPlus</code>
bad case	<code>BadCase</code>	<code>Rotation</code>
rotation	<code>rotPicks</code> (with <code>rotY</code>); the upgraded list becomes <code>k :: up</code>	<code>Rotation</code>
LB^+	<code>lbPlus</code> (K3ALG's run), <code>LBPlusOut</code> and <code>LBPlusRun</code> (all runs)	<code>LBPlus</code>
K3ALG	<code>EFX.K3.algoSpec</code> (specification), <code>EFX.K3.algoC</code> (counted program), <code>EFX.K3.algo</code> (its value)	<code>K3Algo</code> , <code>K3Cost</code>
serial dictatorship (Cor. 2)	<code>EFX.SDRun</code> (all runs), <code>EFX.Inst.serialDict</code> (one run, with <code>EFX.favorite</code>)	<code>K3Extras</code>
scaling (Cor. 1)	<code>EFX.ratScale</code> , with <code>EFX.denProd</code> (the product of the denominators); <code>EFX.K3.algoRat</code>	<code>K3Extras</code>
oracle, w (Cor. 4)	<code>EFX.K3.askC</code> (one call), <code>EFX.K3.surrogate</code> and <code>EFX.K3.surrogateC</code> (w), <code>EFX.K3.algoOrd</code> and <code>EFX.K3.algoOrdC</code>	<code>K3Real</code>

Table 11. Exhaustive certification. Every row covers every ranking profile of every listed core, and its certificates were re-checked from the raw EFX₀ definition.

Claim and scope	Size
EFX ₀ exists, every 3-relevant instance, $n \leq 6$	251 + 3,182 connected cores
EFX ₀ exists, every 3-relevant instance, $n \leq 7$	adds $n = 7$, by the rows on $n = 7$ below
Theorem 2, every core with $n \leq 6$, connected or not	certificates, components combined
Theorem 2, connected cores, $n = 7$, $11 \leq m \leq 13$	3,103, 541 and 37 cores ($m = 11, 12, 13$)
Theorem 2, connected cores, $n = 8$, $m \in \{14, 15\}$	1,232 and 52 cores
LB never fails, cores with $n \leq 6$	3,436 connected, 131 disconnected
LB never fails, connected cores with $n = 7$	41,170 cores, 1.15×10^{10} core-profile pairs
LB never fails, connected cores, $n = 8$, $m \geq 13$	12,763 cores, 2.14×10^{10} pairs

by a second, independent method (a second generator, or counting orbits under relabelling). For each core and each of its 6^n strict ranking profiles, an EFX₀ allocation was found, by a SAT solver with counterexample-guided refinement or by construction LB below, and stored. A separate checker, which uses no SAT solver, re-checks the stored allocations from the raw EFX₀ definition, and that they cover every profile. Table 11 summarizes the results; with Proposition 3, the components argument and the cores with $m = 2n$, they give Theorem 1 for $n \leq 7$ independently of Section 5.

LB is the repository’s earlier construction: Phase 1 with a lookahead rule for insertion steps, the same upgrades, and a search over owners instead of the rotation. The rows on LB say that LB never fails for the labelling of each core stored in the certificates, since LB breaks ties by index; the certificates themselves, which consist of LB’s outputs, confirm Theorem 2 for every labelling. Whether LB never fails in general is open; LB⁺’s rotation makes that question unnecessary.

F.2 Tests of LB⁺ and K3ALG (measurement)

A literal implementation of LB⁺ was run 5,844,263,904 times: on every core with $n \leq 6$ under every ranking profile and every sequence of insertion choices, with random choices on larger cores, and on random instances that are not cores. It asserts every lemma of Section 5 along the way and checks every output against the raw definition: 1,327,101 runs

needed the rotation, and there were 0 failures. An independent Python implementation gives identical tallies for $n \leq 5$. K3ALG itself returned an EFX_0 allocation on all 152,071,632 ranking profiles of all cores with $n \leq 6$. These are measurements of the implementations, not part of the proof.

F.3 Reproducibility

Version. The files named here are those of commit `c6d1871` of the repository [9], the commit cited in the bibliography; the certificates, scripts and Lean sources are unchanged between that commit and the version of this paper. The repository has no release tag.

Scripts. Enumeration of the connected cores: `src/cores_nauty.py`, with `nauty`’s `genbg`, cross-checked against the independent generator of `src/frontier.py`; orbit counting: `tools/check_enum.py`. Search with a SAT solver and counterexample-guided refinement: `src/frontier.py`, driven by `src/run7.py` for $n \geq 7$. Construction LB: `src/construct.c` with `src/construct_run.py`. The SAT-free certificate checker: `tools/check_certs.py`. Tests: `src/lbplus.c` and the independent `src/lbplus.py` for LB^+ , and `k3/k3algo.py` for K3ALG.

Certificate format. Each certificate file, `results/certs_*.json.gz`, is a gzip-compressed JSON list with one record per connected core: the numbers `n` and `m`; `pi`, the number of private goods; `sets`, the three relevant goods of each agent; `mode`, a label (for the SAT runs, the shape of allocation searched for); and `allocations`, a list of allocations, each giving the agent of every good, which together must cover all 6^n ranking profiles. The checker decides, for every stored allocation, in which profiles it is EFX_0 , from the raw definition under three balanced realizations of the rankings, and checks that every profile is covered; with `--require-d` it also checks that every stored allocation has at most one bundle of more than two goods.

Machines and running times. The logs in `results/` record the commands, their dates, the number of CPUs (four, or two for some re-checks) and some wall-clock times. For instance, the SAT search over the 3,103 connected cores with $n = 7$ and $m = 11$ took 684s on four CPUs (`results/frontier7_11.log`); K3ALG on every ranking profile of every core with $n \leq 6$ took 4,469s on four CPUs (`results/k3_certs_2_6.log`); and construction LB on the connected cores with $n = 8$ and $m \geq 13$ took

about 7 h on four CPUs (the repository's ledger). The processor model and the memory are not recorded. The measurements of Section 6.5 used Python 3.11.15 (`results/k3_timing.log`).

Re-checking. `python tools/check_certs.py file --require-d` re-checks one certificate file. The repository's continuous integration re-checks some of the certificates and the orbit counts on every pull request; the larger files were re-checked locally, as the logs record.